



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

Crash Course

(ONE SHOT Revision)

Unit-5

Pdf Notes | AKTU PYQs | Imp. Questions



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

AKTU B.Tech I-Year

Syllabus Unit-5

Pointers: Introduction, Declaration, Applications, Introduction to Dynamic Memory Allocation (Malloc, Calloc, Realloc, Free), String and String functions, Use of Pointers in Self-Referential Structures, Notion of Linked List (No Implementation)

File Handling: File I/O Functions, Standard C Preprocessors, Defining and Calling
Macros and Command-Line Arguments.



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-5

Lecture-1

Topics to be Covered

- Preprocessor Macro
- Conditional Compilation
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Video Lectures | Pdf Notes

AKTU PYQs

1. Write short note on macro.(AKTU 2023-24)
2. What is macro? Illustrate the working of macro as a variable and as a function with the help of a suitable example.(AKTU 2022-23)
3. Explain the role of C preprocessor. What is macro? What are the advantages of using macro definitions in a program?(AKTU 2021-22)
4. Why are preprocessor required? Explain any two preprocessor directives.(AKTU 2021-22)
5. Define preprocessor and its usage in programming?(AKTU 2021-22)
6. Write a short note on following preprocessor directives with example:
 - i. Macro Expansion
 - ii. File Inclusion.(AKTU 2019-20)
7. Write macro definitions with arguments for calculation of simple interest and amount. Store these macro definitions in a file called 'interest.h'. Include this file in your program and use the macro definitions for calculating simple interest and amount. (AKTU 2019-20)

AKTU PYQs

8. Explain the syntax and use of the following directives with example. (AKTU 2023-24)

#ifdef

#pragma

#undef

#include

9. Explain conditional compilation. (AKTU 2021-22, 2019-20)

Gateway classes

The C Preprocessor

- Code that we write is translated into object code by the compiler.
- But before being compiled, the code is passed through the C preprocessor. The preprocessor scans the whole source code and modifies it, which is then given to the compiler.

→ Preprocessor removes the comments from the source code.

Some features of preprocessor directives:

- Starts with a # symbol.
- Only one directive on a line.
- No semicolon at the end of a directive.
- Can be placed anywhere in a program (inside or outside functions) usually written at the beginning of a program.
- (vi) A directive is active from the point of its appearance till the end of the program.

#include <stdio.h>
↓
Preprocessor
directive

source file

↓ preprocessor
expanded code

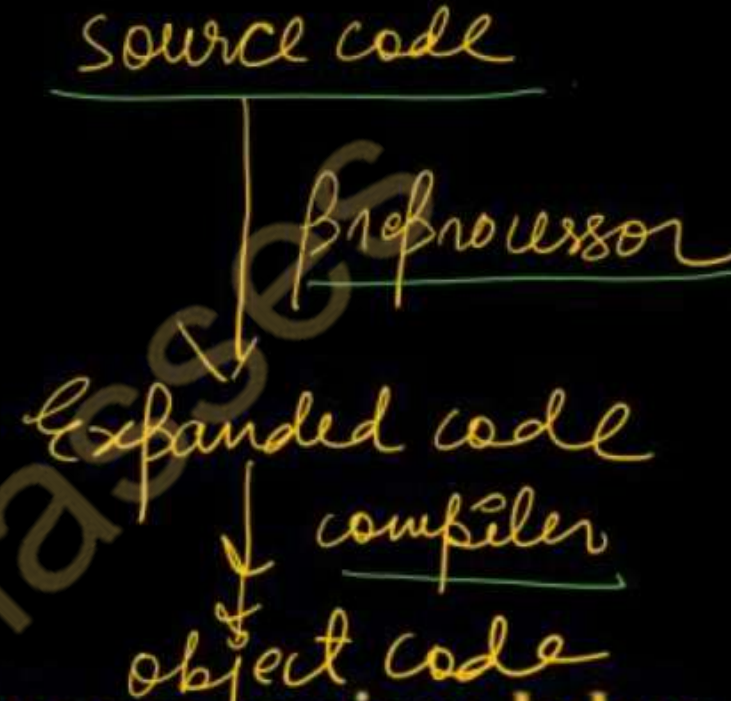
↓ compiler
object code

#include <stdio.h> ✓
↓

A.C
source code

The main functions performed by the preprocessor directives:

1. Simple Macro Substitution
2. Macros with arguments
3. Conditional Compilation
4. Including files



The preprocessor directives that perform these functions are as given below-

- | | |
|--------------------|--------------|
| ① <u>#define</u> ✓ | ⑥ #else |
| ② #if | ⑦ #elif |
| ③ #ifndef | ⑧ #undef |
| ④ #ifdef | ⑨ #endif |
| ⑤ #pragma | ⑩ #include ✓ |

preprocessor

#define

syntax-

Macro

preprocessor
directive

#define macro_name macro_expansion

Macro Template

#define MAX 100
main()
{
int a[MAX];
}

The C preprocessor replaces all the occurrences of macro_name with macro_expansion.
For example-

#define X 100

Macro Template
Macro Expansion

#define TRUE 1

#define FALSE 0

#define PI 3.14159265

#define MAX 100

#define RETIRE AGE 58

Macro as Variable

Example 1:

```
#define UPPER 25
```

```
main( )  
{  
    int i;  
    for ( i = 1 ; i <= UPPER ; i++ )  
        printf ( "\n%d", i );  
}
```

1 2 3 ... 25

#define UPPER 25 This statement is called 'macro definition' or just a 'macro'.

Example 2:

```
#define PI 3.1415
```

```
main( )  
{  
    float r = 6.25 ;  
    float area ; area = PI * r * r ;  
    printf ( "\nArea of circle = %f", area ) ;  
}
```

3.1415

UPPER and PI in the above programs are often called '**macro templates**', whereas, 25 and 3.1415 are called their corresponding '**macro expansions**'.

Example 3:

```

#define AND &&
#define OR ||
main( )
{
    int f = 1, x = 4, y = 90 ;
    if ( ( f < 5 ) AND ( x <= 20 OR y <= 45 ) )
        printf ( "\nYour PC will always work fine..." ) ;
    else
        printf ( "\nIn front of the maintenance man" ) ;
}

```

Example 5:

```

#define AND &&
#define ARANGE ( a > 25 AND a < 50 )
main( )
{
    int a = 30 ;
    if ( ARANGE )
        printf ( "within range" ) ;
    else
        printf ( "out of range" ) ;
}

```

Example 4:

```

#define FOUND printf ( "The Yankee Doodle Virus" ) ;
main( )
{
    char signature ;
    if ( signature == 'Y' )
        FOUND
    else
        printf ( "Safe... as yet !" ) ;
}

```

Points To Remember:

- Macro template and its macro expansion are separated by blanks or tabs.
- A space between # and define is optional.

Some more examples:

1. #define AND && ✓
2. #define OR || ✓
3. #define BEGIN main() {
4. #define END }
5. #define INFINITE while(1)
6. #define NEW_LINE printf("\n");
7. #define ERROR printf("An error has occurred\n");
8. #define MSSG "I understand the use of #define\n"
main()
{
printf(MSSG);
}

Note: If the macro_name appears inside a character constant, string constant or a comment then it is not replaced and is left as it is.

For example

```
#define SUN 2
```

```
#define WINDOWS 3
```

```
printf("Rays of SUN are coming through the WINDOWS");
```

➤ It is also possible to define a macro without any macro expansion.

for example

```
#define TEST
```

will not be replaced with 2/3.

Advantages of using macro definitions in a program (AKTU 2021-22)

1. There are many instances where a constant doesn't reveal its purpose so readily. For example, if the phrase "\x1B[2J" causes the screen to clear. Understanding "CLEARSCREEN" is easier in the middle of your program than understanding "\x1B[2J"

```
#define CLEARSCREEN "\x1B[2J"
```

All occurrences of CLEARSCREEN would automatically be replaced by "\x1B[2J" before compilation.

2. Suppose a constant like 3.1415 appears many times in your program. This value may have to be changed some day to 3.141592. Then writing

(Macro as Variable)

```
#define PI 3.141592
```

you only need to make one change, in the #define directive itself and the change will be made automatically to all occurrences of PI before the beginning of compilation.

Macros with Arguments

#define directive can also be used to define macros with arguments.

Syntax : #define macro_name(arg 1, arg2,) macro_expansion

Macro acting as
a function

Example:

```
#define AREA(x) ( 3.14 * x * x )
```

```
main( )
```

```
{
```

```
float r1 = 6.25, r2 = 2.5, a ;
```

```
a = AREA( r1 ) ;
```

```
printf ( "\nArea of circle = %f", a ) ;
```

```
a = AREA( r2 ) ;
```

```
printf ( "\nArea of circle = %f", a ) ;
```

```
}
```

output: Area of circle = 122.656250

Area of circle = 19.625000

Preprocessor expands AREA(x) into (3.14 * x * x)

Macro Template

Macro Expansion

expanded as $3.14 * r_1 * r_1$;

expanded as $3.14 * r_2 * r_2$;

- x in macro template $AREA(x)$ is an argument that matches x in the macro expansion $(3.14 * x * x)$
- $AREA(r1)$ in the program causes the variable $r1$ to be substituted for x . Thus the statement $AREA(r1)$ is equivalent to: $(3.14 * r1 * r1)$.
- After the above source code has passed through the preprocessor, what the compiler gets to work on

will be this:

compiler
↓

```
main( )  
{ float r1 = 6.25, r2 = 2.5, a ;  
  a = 3.14 * r1 * r1 ;  
  printf ( "Area of circle = %f\n", a ) ;  
  a = 3.14 * r2 * r2 ;  
  printf ( "Area of circle = %f", a ) ;  
}
```


Points To Remember:

- Do not leave a blank between the macro template and its argument while defining the macro. For example, there should be no blank between AREA and (x) in the definition.
- #define AREA(x) (3.14 * x * x) If we were to write AREA (x) instead of AREA(x), the (x) would become a part of macro expansion, which we certainly don't want.
- Template would be expanded to (r1) (3.14 * r1 * r1) which won't run. Not at all what we wanted.
- The entire macro expansion should be enclosed within parentheses.

X

```
#define AREA (x) (3.14 * x * x)
```

perceived as Macro Template

macro expansion

Example:

```
#define SQUARE(n) n * n
```

```
main( )
```

```
{
```

```
    int j ;
```

```
    j = 64 / SQUARE ( 4 ) ;
```

```
    printf ( "j = %d", j ) ;
```

```
}
```

(AKTU)

$$j = 64 / 4 * 4 ;$$

$$j = 16 * 4$$

$$j = 64$$

$$j = 64 / (4 * 4)$$

$$\Rightarrow 64 / 16 = 4$$

Find the output of the following code:

(AKTU 2023-24)

Find the output of following code: -

```
#include<stdio.h>
#define a (2*2+2)
void main ()
{
int b,c;
b=2/a; // 4
c=b+4; // 8
printf("Value of variable b and c are %d%d respectively ", b,c);
}
```

$$\underline{b} = 2/a \Rightarrow \underline{b} = 2/2 \times 2 + 2 \\ = 1 \times 2 + 2 = \underline{4}$$

$$b = 2/(2 \times 2 + 2) \quad \downarrow \downarrow \\ 2/6 \quad \underline{b=0} \quad \downarrow \downarrow \\ \quad \quad \quad 4 \quad 8$$

/*Program to print greater number between two numbers using macro*/

```
#include <stdio.h>
```

```
#define Max(a,b) ((a>b)? a : b)
```

⇒ Macro with argument definition

```
int main()  
{
```

```
    int a, b, big;
```

```
    printf("Enter two numbers");
```

```
    scanf("%d%d", &a, &b);
```

```
    big = Max(a, b);
```

⇒ ((a>b)? a : b);

```
    printf("The greater number is = %d", big);
```

```
    return 0;
```

```
}
```


Write macro definitions with arguments for calculation of simple interest and amount. Store these macro definitions in a file called 'interest.h'. Include this file in your program and use the macro definitions for calculating simple interest and amount. (AKTU 2019-20)

A.C

```
#include <stdio.h>
#include "interest.h"
int main()
{
    float p, t, r, si, amnt;
    printf("Enter principal, time, and rate of interest : ");
    scanf("%f%f%f", &p, &t, &r);
    si = SI(p, t, r);  $\Rightarrow (p \times t \times r) / 100.0;$ 
    amnt = AMOUNT(si, p);  $\Rightarrow si + p$ 
    printf("\nSimple Interest : %f\nAmount : %f", si, amnt);
    return 0;
}
```

create a file with
name interest.h in the
same folder as A.C

```
#define SI(a,b,c) ((a*b*c)/100.0)
#define AMOUNT(si,p) (si+p)
```

Difference between macro and function (AKTU)

Macro	Function
It is not <u>compiled</u> , it is <u>pre-processed</u> .	It is not pre-processed but it is compiled.
Macros do not check for compilation error which leads to unexpected output.	Function checks for compilation error and there is a less chance of unexpected output.
Code length is increased.	Code length remains same.
Macros are faster in execution than function.	Functions are bit slower in execution.
Before compilation process the macro name is replaced by the macro value.	In a function call, transfer of control takes place.
Macros are useful when a small piece of code used multiple times in a program.	Functions are helpful when a large piece of code repeated number of times.

File Inclusion:

(AKTU)

Actually there exist two ways to write #include statement. These are:

- ✓ #include "filename"
- ✓ #include <filename>

The meaning of each of these forms is given below:

- **#include "goto.c":** This command would look for the file goto.c in the current directory as well as the specified list of directories as mentioned in the include search path that might have been set up.
- **#include <goto.c>:** This command would look for the file goto.c in the specified list of directories only.

Imp Conditional Compilation

We can, if we want, have the compiler skip over part of a source code by inserting the preprocessing commands #ifdef and #endif

Example:

```
#ifdef macroname  
statement 1 ;  
statement 2 ;  
statement 3 ;  
#endif
```

if defined

✓
if (—)
{
==
}
}

If macroname has been #defined, the block of code will be processed as usual; otherwise not.

Where would #ifdef be useful?

Example 1:

It often happens that a program is changed at the last minute to satisfy a client rewriting some part of source code and deleting the old code. But veteran programmers are familiar with the clients who change their mind and want the old code back again just the way it was. Therefore the solution is to use conditional compilation as shown below.

```
main( )  
{  
    #ifdef OKAY  
        statement 1 ;  
        statement 2 ;  
    #endif  
    statement 3 ;  
}
```

Here, statements 1, 2 would get compiled only if the macro OKAY has been defined.

Example 2:

Suppose an organization has two different types of computers and you are expected to write a program that works on both the machines.

```
main( )  
{  
    #ifdef INTEL  
        code suitable for a Intel PC  
    #else  
        code suitable for a Motorola PC  
    #endif  
    code common to both the computers  
}
```


#if and #elif Directives

The #if directive can be used to test whether an expression evaluates to a nonzero value or not. If the result of the expression is nonzero, then subsequent lines upto a #else, #elif or #endif are compiled, otherwise they are skipped.

```
main( )  
{  
    #if TEST <= 5  
    statement 1 ;  
    statement 2 ;  
    #else  
    statement 4 ;  
    statement 5 ;  
    #endif  
}
```

#elif

#ifdef

#else

#endif

if TEST <= 5

elif TEST > 5

else

endif

#undef Directive:

In order to undefine a macro that has been earlier **#defined**, the directive, **#undef macro template** can be used. Thus the statement,

#undef PENTIUM

would cause the definition of PENTIUM to be removed from the system.
All subsequent **#ifdef PENTIUM** statements would evaluate to false.

✓ #define X 100
#undef X

#pragma (once asked in AKTU)

- **#pragma** is used to provide additional information to the compiler in C. This is used by the compiler to provide some special features.
- pragmas are compiler dependent. Not all the pragma directives are supported by all the compilers.

Directive	Description
<u>#pragma startup</u>	Before the execution of main(), the function specified in pragma is needed to run.
<u>#pragma exit</u>	Before the end of program, the function specified in pragma is needed to run.
<u>#pragma warn</u>	Used to hide the warning messages.

```
#include <stdio.h>
void func1();
void func2();
#pragma startup func1 ✓
#pragma exit func2 ✓
void func1()
{
    printf("Inside func1()\n");
}
void func2()
{
    printf("Inside func2()\n");
}
int main()
{
    printf("Inside main()\n");
    return 0;
}
```


The following table lists all the preprocessor directives in C:

Preprocessor Directives	Description
#define	Used to define a macro
#undef	Used to undefine a macro
#include	Used to include a file in the source code program
#ifdef	Used to include a section of code if a certain macro is defined by #define
#ifndef ✓	Used to include a section of code if a certain macro is not defined by #define
#if	Check for the specified condition
#else	Alternate code that executes when #if fails
#elif	Combines else and if for another condition check
#endif	Used to mark the end of #if, #ifdef, and #ifndef



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-5: Lecture-2 [PART-1+PART-2]

Topics to be Covered

- Pointers
- Linked List
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Address Operator(&)

- An address operator '&' returns the address of a variable when placed before it.
- Addresses are just whole numbers. These addresses may be different each time you run your program, it depends on which part of memory is allocated by operating system for this program.

*/*Program to print address of variables using address operator*/*

```
#include<stdio.h>
```

```
main( )
```

```
{
```

```
    int a=30;
```

```
    float b=1500.50;
```

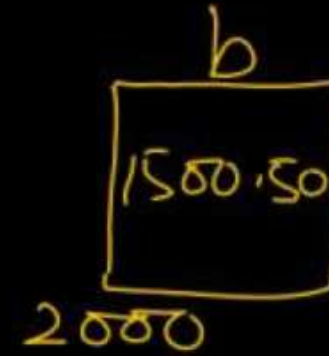
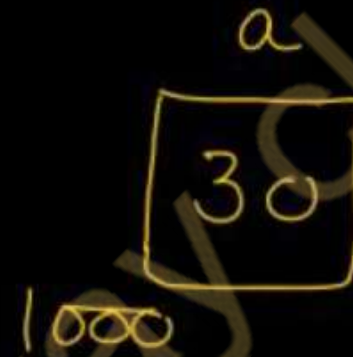
```
    printf ("Value of a =%d, Address of a= %u\n", a, &a);
```

```
    printf ("Value of b=%f, Address of b= %u\n", b, &b);
```

```
}
```

Output:

&



output

value of a = 30, Address of a = 1000

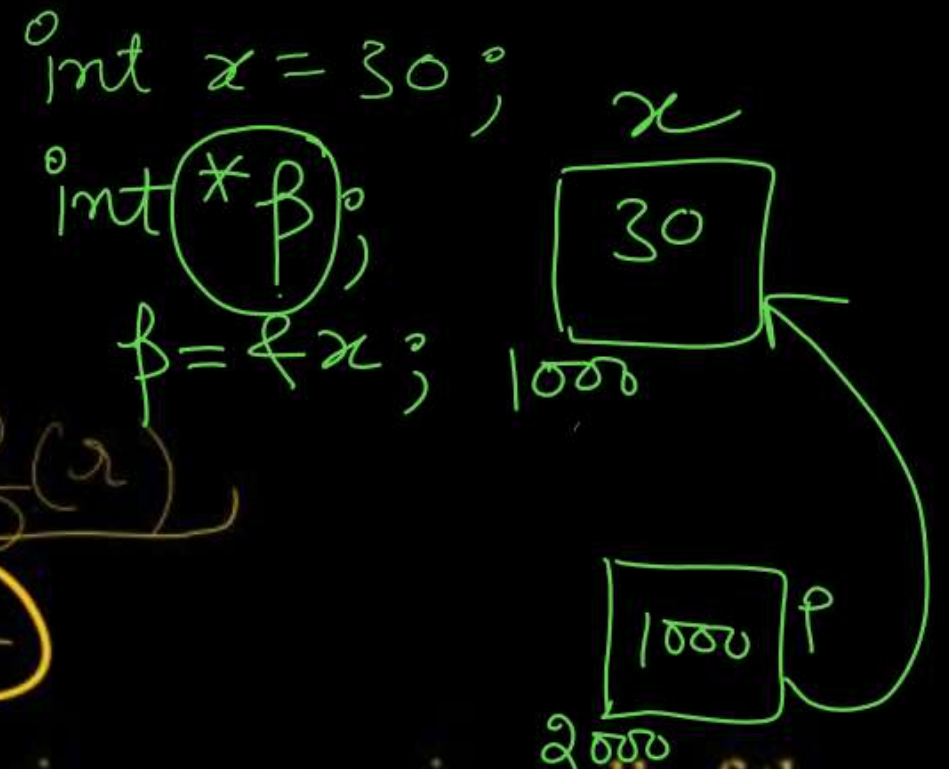
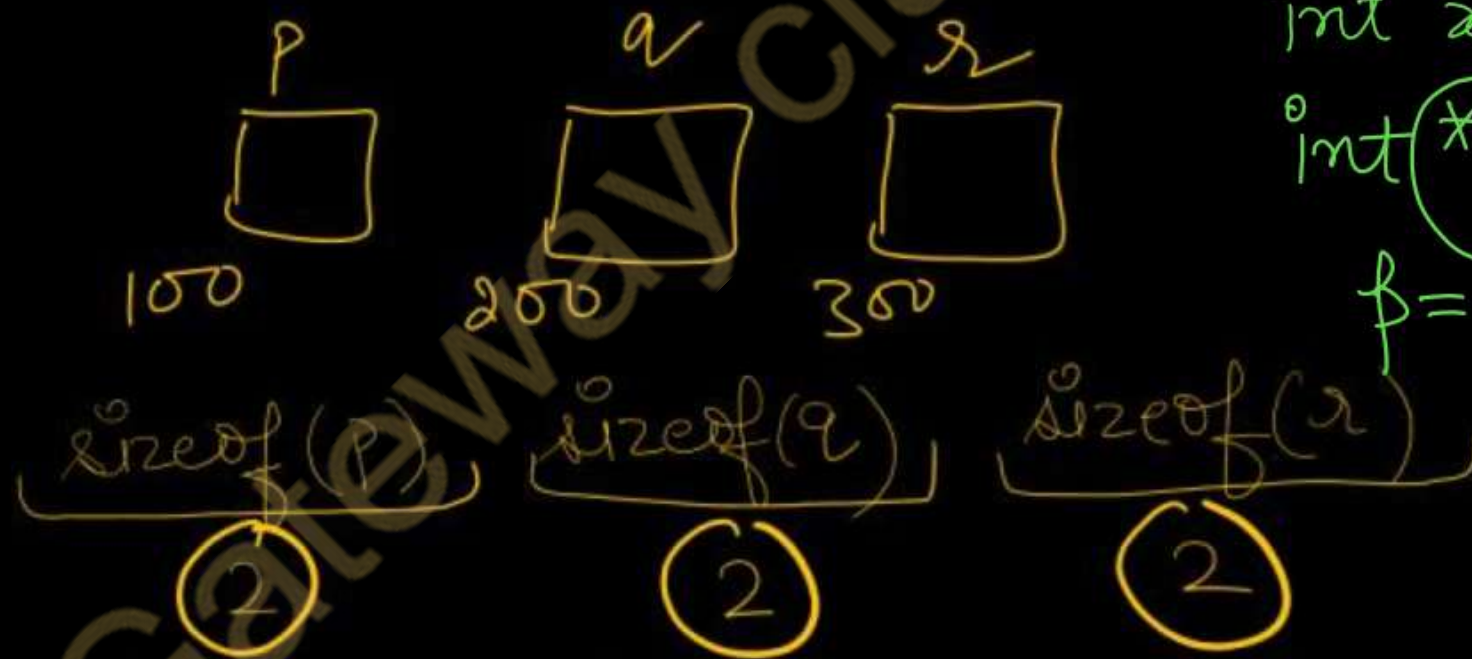
value of b = 1500.50, Address of b = 2000

Pointer

A pointer is a variable that stores memory address. *of another variable of same type.*
Like all other variables it also has a name, has to be declared and occupies some space in memory.
It points to a particular location in memory by storing the address of that location.

Declaration of pointer variables

- ① `int *p;`
- ② `float *q;`
- ③ `char *r;`



➤ All pointers irrespective of their base type will occupy same space in memory since all of them contain addresses only.

Assigning address to pointer variables

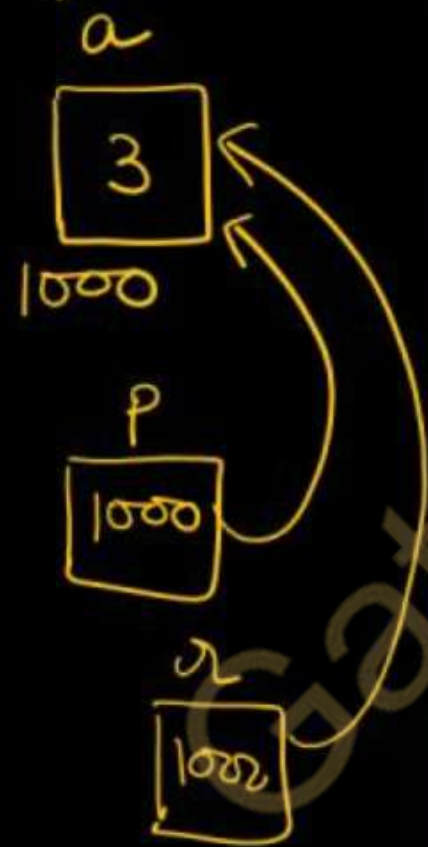
- When we declare a pointer variable it contains garbage value (it may be pointing anywhere in the memory. So, we should always assign an address before using it in the program.
- The use of an unassigned pointer may give unpredictable results and even cause the program to crash.
- Pointers may be assigned the address of a variable using assignment statement.

For example-

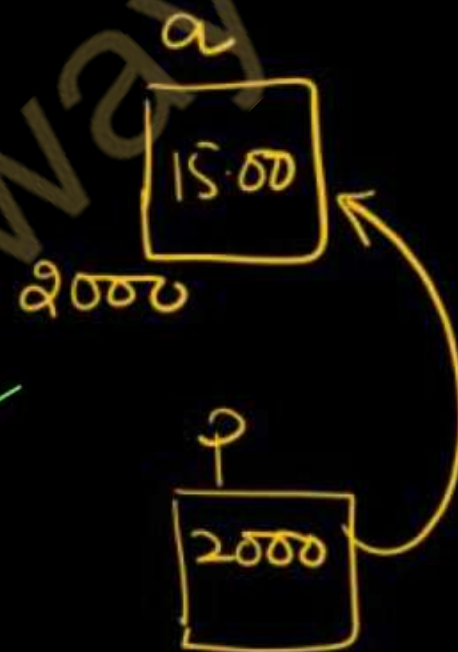
int *p;
int a = 3; *p;
p = &a; ✓

p = &a; X

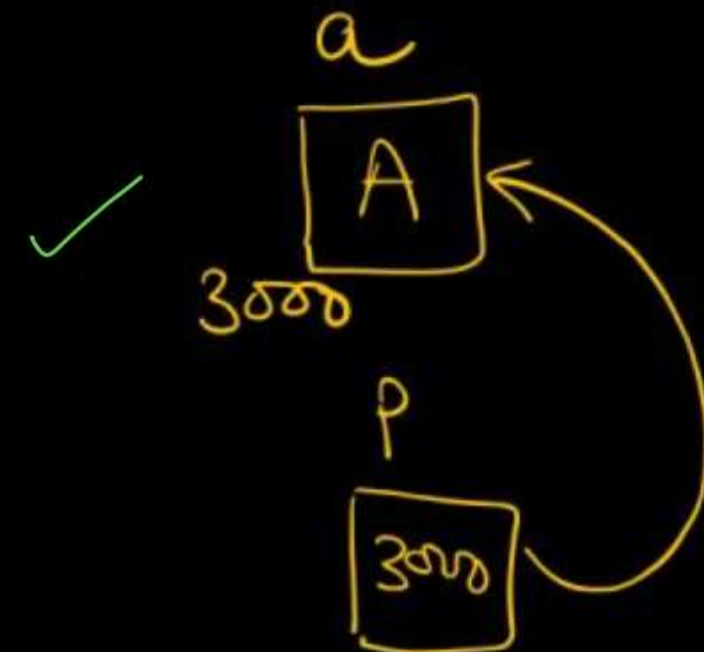
p = 3 X
{ p = &a; ✓
p = 0; ✓
p = NULL; ✓
p = &p; ✓



float a = 15.00; *p;
p = &a; ✓
a
15.00
2000
p
2000



char a = 'A'; *p;
p = &a; ✓



Dereferencing pointer variables

- We can also access a variable using pointers by using indirection operator(*). By placing * before a pointer variable, we can access the variable whose address is stored in the pointer.

/*Program to dereference pointer variables */

```
#include<stdio.h>
```

```
main( )
```

```
{
```

```
int a=87;
```

```
float b=4.5;
```

```
int *p1=&a;
```

```
float *p2=&b;
```

```
printf("Value of p1 = Address of a = %u\n",p1); // 1000
```

```
printf("Value of p2 = Address of b = %u\n",p2); // 2000
```

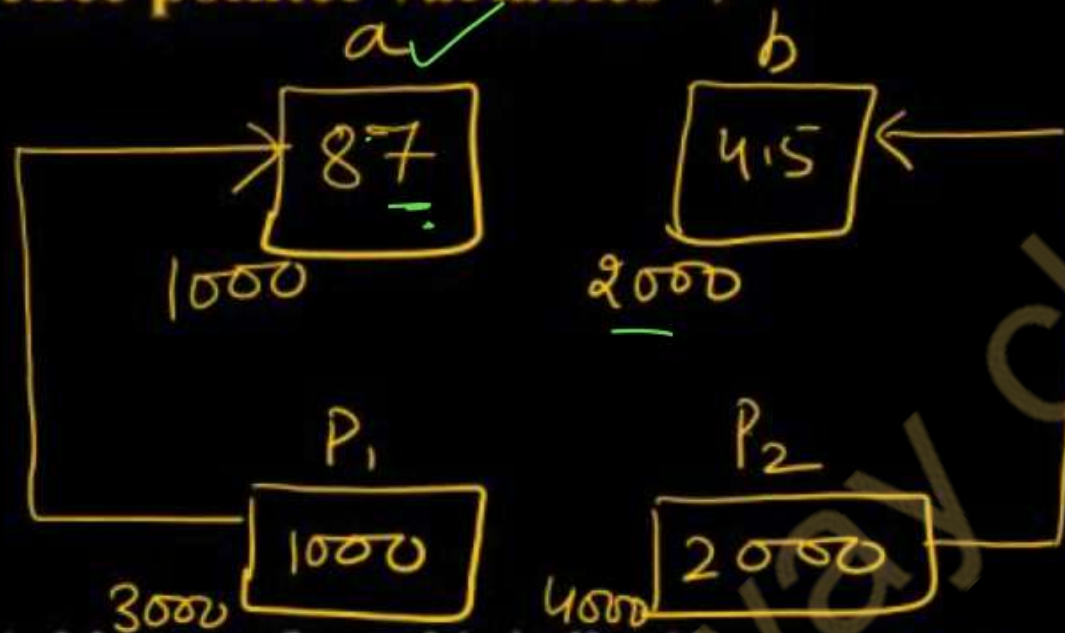
```
printf("Address of p1 %u \n", &p1); // 3000
```

```
printf("Address of p2 %u \n", &p2); // 4000
```

```
printf("Value of a = %d %d %d \n",a,*p1,*(&a)); // 87
```

```
printf("Value of b = %f %f %f \n", b, *p2, *(&b)); // 4.5
```

```
}
```



*p₁ means
value at address stored in p₁

address stored in p₁ = 1000

Value at 1000 = 87

*p₁ ⇒ 87

dereferencing of a pointer

/*Program to print the size of pointer variable and size of value dereferenced by that pointer*/

```
#include<stdio.h>
```

```
main ( )
```

```
(
```

```
char a = 'x' , *p1=&a;
```

```
int b=12, *p2=&b;
```

```
float c=12.4, *p3=&c;
```

```
double d=18.3, *p4=&d;
```

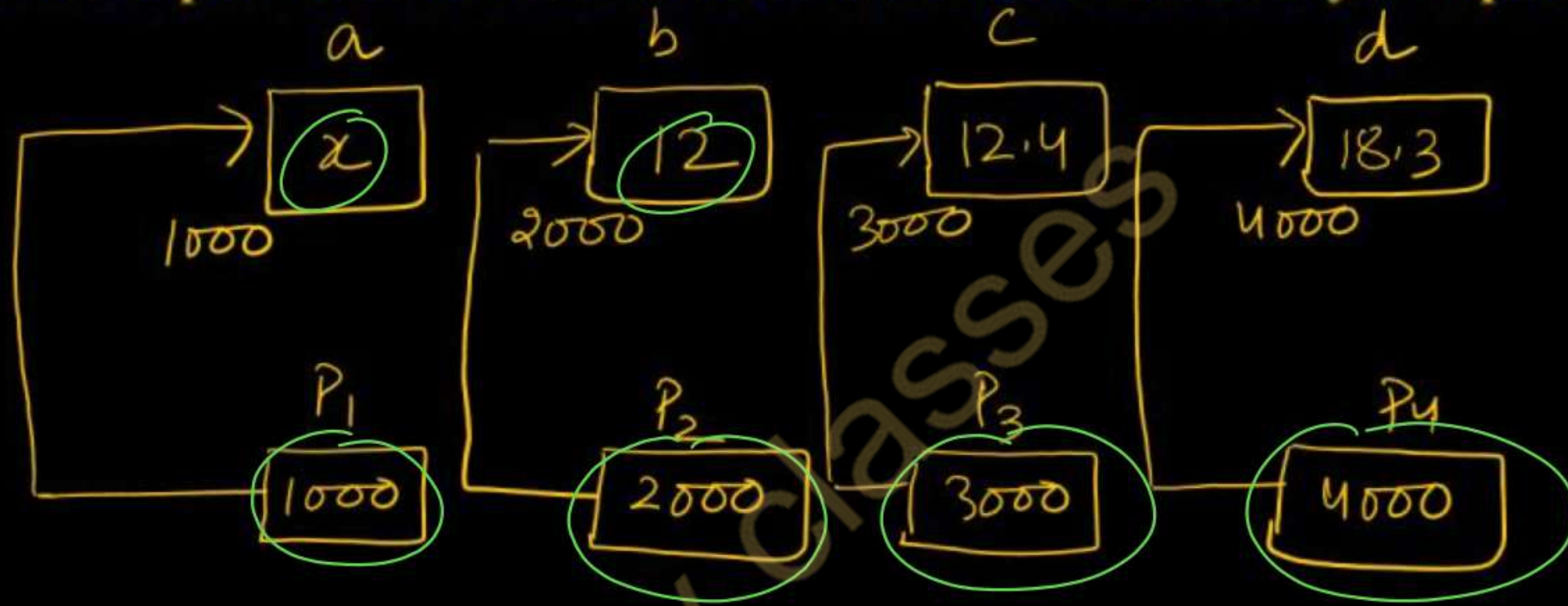
```
printf("sizeof(p1) = %d , sizeof(*p1) = %d\n", sizeof(p1), sizeof(*p1)); // 2, 1
```

```
printf("sizeof(p2) = %d , sizeof(*p2) = %d\n", sizeof(p2), sizeof(*p2)); // 2, 2
```

```
printf("sizeof(p3) = %d , sizeof(*p3) = %d\n", sizeof(p3), sizeof(*p3)); // 2, 4
```

```
printf ("sizeof(p4) = %d , sizeof(*p4) = %d\n", sizeof(p4), sizeof(*p4)); // 2, 8
```

```
}
```



Imp **Pointer Arithmetic**

All types of arithmetic operations are not possible with pointers. The only valid operations that can be performed are as-

- (1) Addition of an integer to a pointer and increment operation.
- (2) Subtraction of an integer from a pointer and decrement operation
- (3) Subtraction of a pointer from another pointer of same type.

The arithmetic operations that can never be performed on pointers are-

1. Addition, multiplication, division of two pointers.
2. Multiplication between pointer and any number.
3. Division of a pointer by any number.
4. Addition of float or double values to pointers.

/*Program to show pointer arithmetic */

```
#include<stdio.h>
```

```
main()
```

```
{  
int a=5, *pi=&a;
```

```
char b='x', *pc=&b;
```

```
float c=5.5, *pf=&c;
```

```
printf("Value of pi = Address of a = %u\n", pi); // 1000
```

```
printf("Value of pc = Address of b = %u\n", pc); // 2000
```

```
printf("Value of pf = Address of c = %u\n", pf); // 3000
```

```
pi++;  $\rightarrow P_i = P_i + 1 \Rightarrow P_i = 1000 + 2 \Rightarrow 1002$ 
```

```
pc++;  $\rightarrow P_c = P_c + 1 \Rightarrow 2000 + 1 \Rightarrow 2001$ 
```

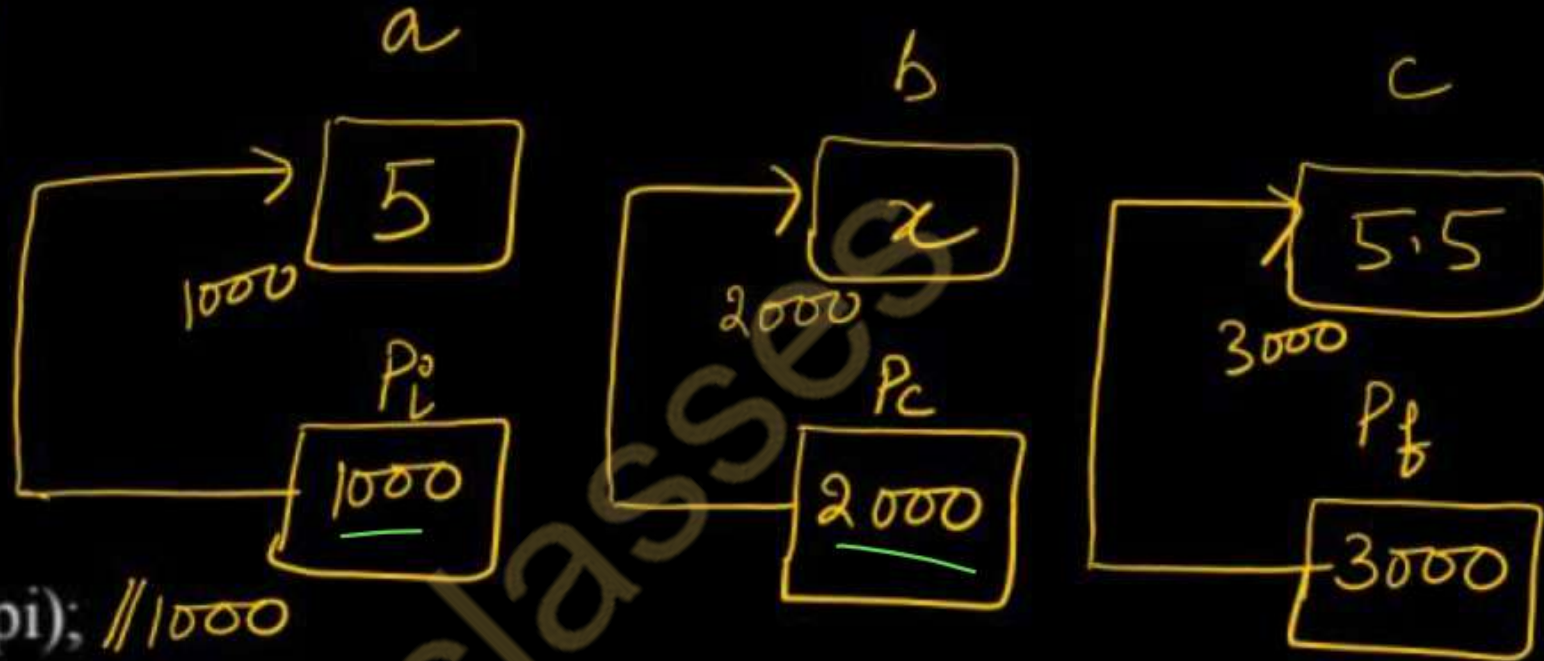
```
pf++;  $\Rightarrow P_f = P_f + 1 \Rightarrow 3000 + 4 = 3004$  for char pointer
```

```
printf("Now value of pi = %u\n", pi); // 1002
```

```
printf("Now value of pc = %u\n", pc); // 2001
```

```
printf("Now value of pf = %u\n", pf); } // 3004
```

int $\rightarrow$ 2 bytes
char $\rightarrow$ 1 byte
float $\rightarrow$ 4 byte

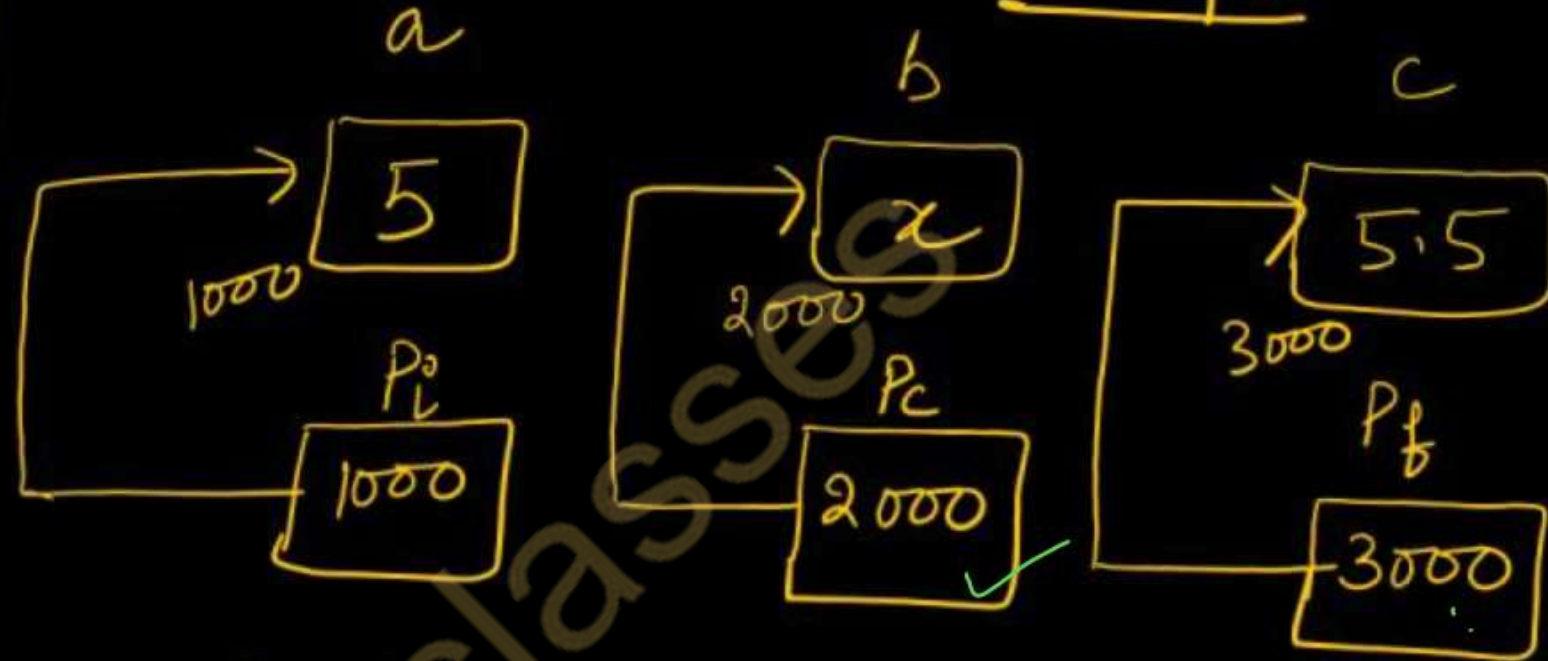


2 is added because P_i is an integer pointer.

/*Program to show pointer arithmetic */ (Imp)

```
#include<stdio.h>
main()
{
int a=5, *pi=&a;,
char b='x', *pc=&b;
float c=5.5, *pf=&c;
```

int $\rightarrow$ 2 bytes
char $\rightarrow$ 1 byte
float $\rightarrow$ 4 byte



$$pi = pi + 2;$$

$$pi = pi - 3;$$

$$pc = pc + 2;$$

$$pc = pc - 3;$$

$$pf = pf + 2;$$

$$pf = pf - 3;$$

```
printf("Now value of pi = %u\n", pi);
```

```
printf("Now value of pc = %u\n", pc);
```

```
printf("Now value of pf = %u\n", pf);}
```

$$pi = pi + 2 \Rightarrow 1000 + (2 \times 2) \Rightarrow 1004$$

$$pi = pi - 3 \Rightarrow 1004 - (3 \times 2) \Rightarrow 998$$

$$pc = pc + 2 \Rightarrow 2000 + (2 \times 1) \Rightarrow 2002$$

$$pc = pc - 3 \Rightarrow 2002 - (3 \times 1) \Rightarrow 1999$$

$$pf = pf + 2 \Rightarrow 3000 + (2 \times 4) \Rightarrow 3008$$

$$pf = pf - 3 \Rightarrow 3008 - (3 \times 4) \Rightarrow 2996$$

/*Program to understand the postfix/prefix increment/decrement in a pointer variable */

```
#include<stdio.h>
```

```
main()
```

```
{
```

```
int a=5;
```

```
int *p;
```

```
p=&a;
```

```
printf("Value of p =%u\n",p); // 1000
```

```
printf("Value of P =%u\n",++p); // 1002 (First incremented then printed)
```

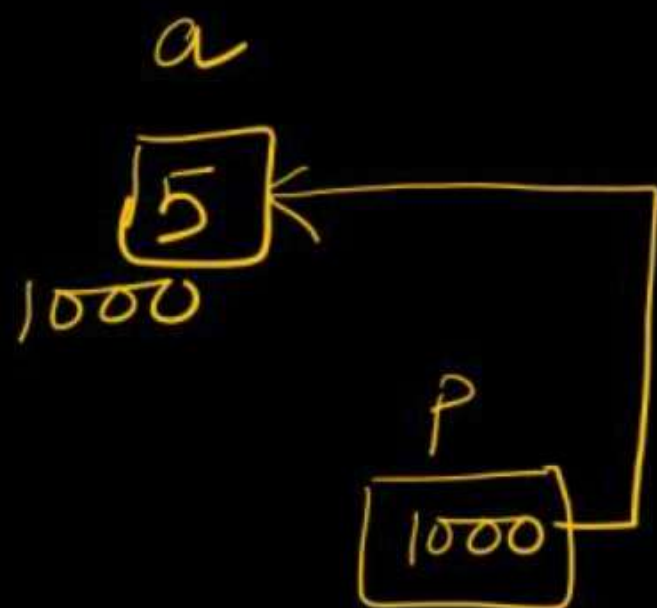
```
printf("Value of p = %u\n",p++); // 1002 (First printed then become 1004)
```

```
printf("Value of p = %u\n",--p); // 1002 (First decrement, become 1002 then printed)
```

```
printf("Value of p = %u\n",p--); // 1002 (First printed then become 1000)
```

```
printf("Value of p = %u\n",p); // 1000
```

```
}
```



Subtraction of a pointer from another pointer

Subtraction of a pointer from another pointer of same base type returns an integer, which denotes the number of elements between two pointers.

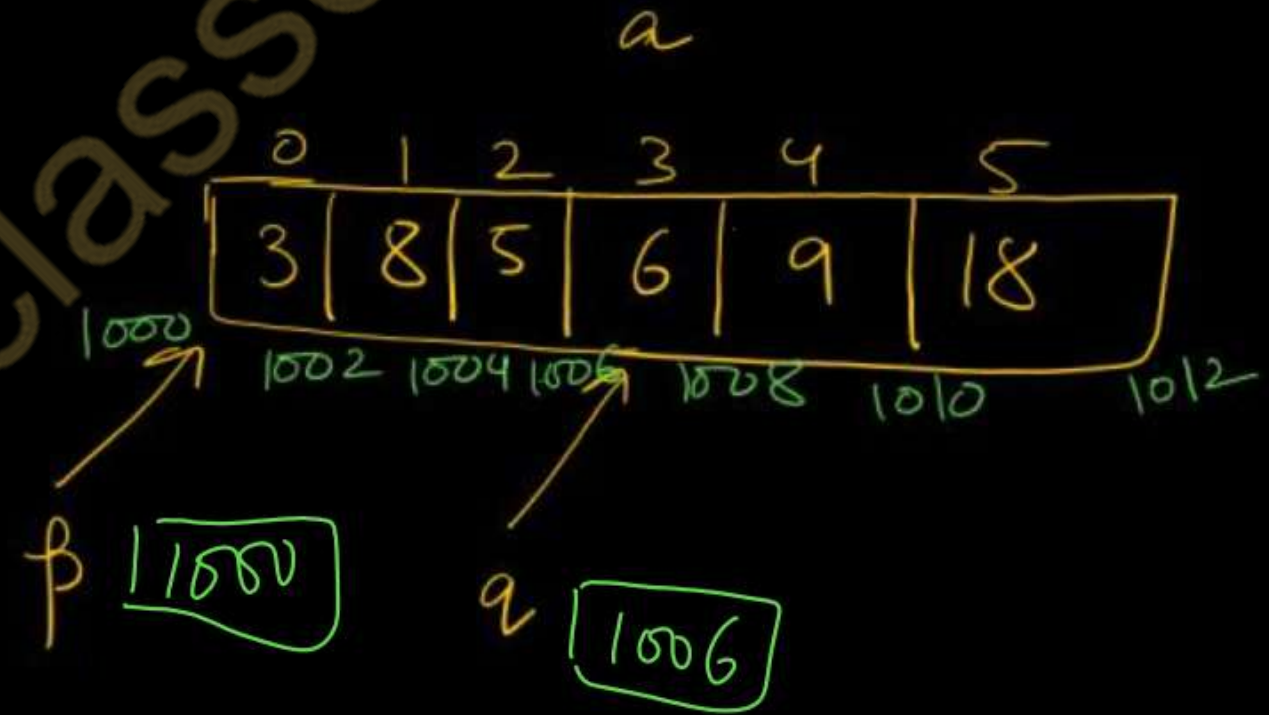
```
int x;  
int a[] = {3, 8, 5, 6, 9, 18};  
int *p, *q;  
p = &a[0];  
q = &a[3];  
x = q - p;  
printf("%d", x);
```

$$x = q - p$$

$$1006 - 1000$$

$$= 6 / 2 \text{ bytes}$$

$$= 3$$



→ as integer takes 2 bytes.

Comparison between Pointers

```
int a[] = {8, 5, 6, 3, 2, 1};
```

```
int *p, *q;
```

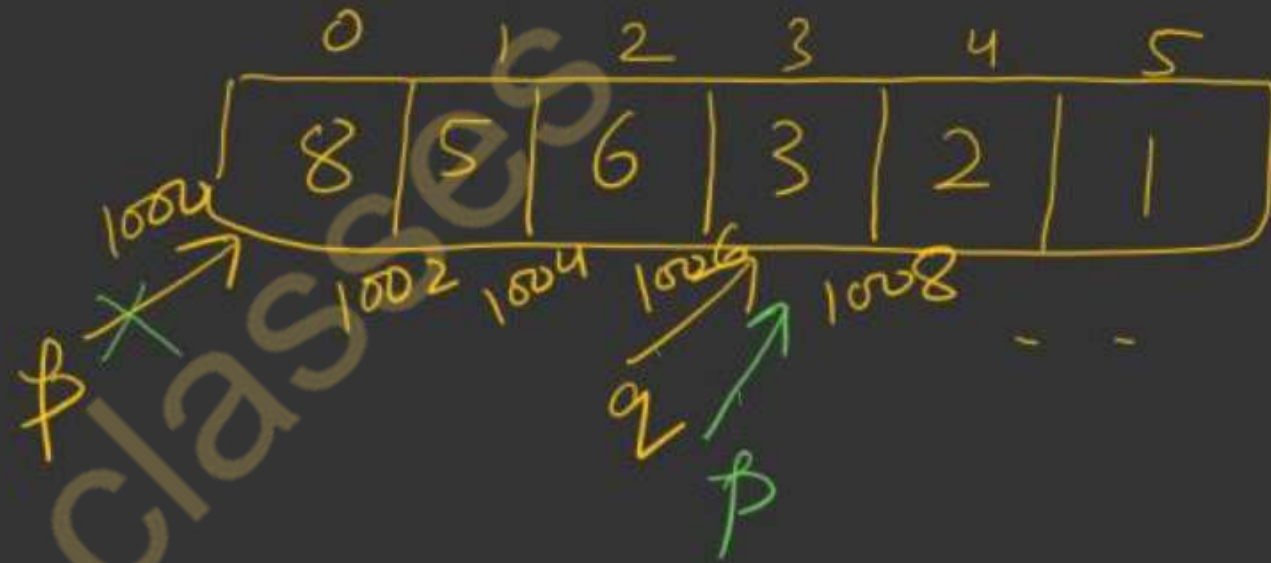
```
p = &a[0];
```

```
q = &a[3];
```

```
if (p == q)
```

```
printf("Both pointers are pointing to same memory location");
```

```
else printf("Both pointers are pointing to different memory location");
```

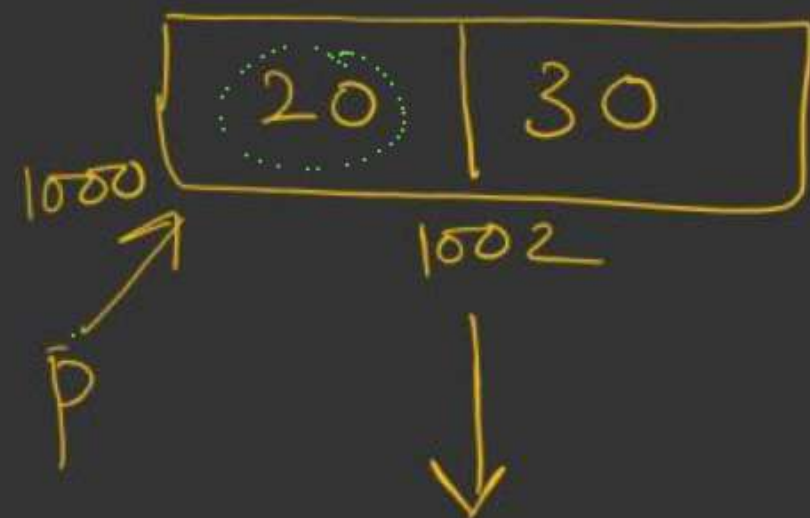


Working of (++) with dereferencing operator (*)

Note Precedence of ++ and (*) is same and associativity is right to left.

$$\textcircled{1} \quad x = *(\beta++);$$

$$\Downarrow$$
$$x = *(p++);$$

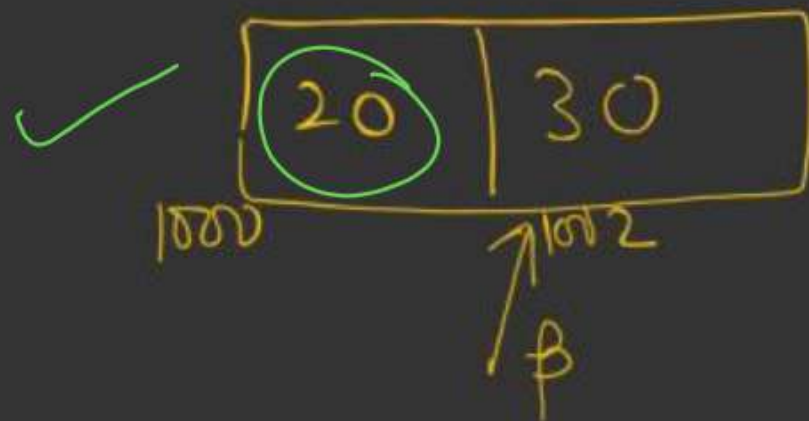


$$x = *p;$$

$$p = p + 1;$$

$$\checkmark \quad x = 20$$

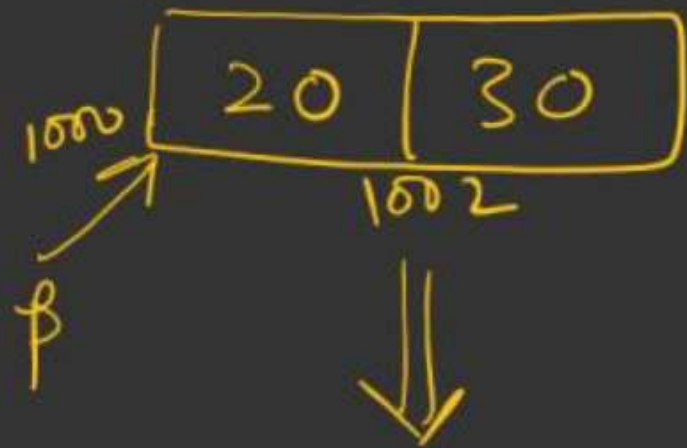
$$\checkmark \quad p = 1000 + 2 = 1002$$



Working of (++) with dereferencing operator (*)

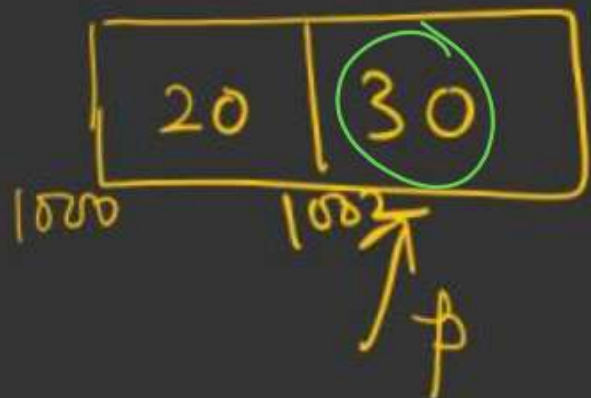
② $x = *++p;$

$\Downarrow$
 $x = *(\underline{++p});$ — $\left[\begin{array}{l} p = p + 1; \\ x = *p; \end{array} \right.$



$$p = 1000 + 1 = \underline{1002}$$

$$\checkmark x = 30$$



Working of (++) with dereferencing operator (*)

(3) $x = \underline{++^*p};$

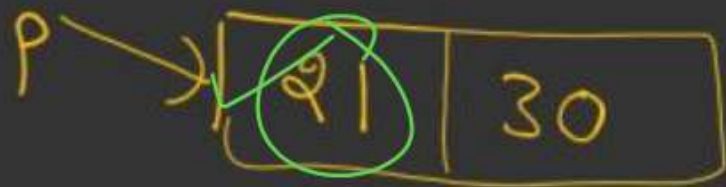
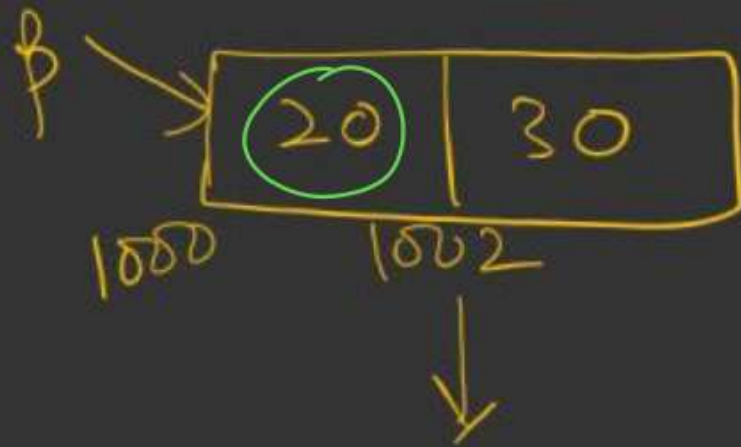
$x = \underline{++(\underline{*p})};$

$$\begin{cases} *p = *p + 1; \end{cases}$$

$$\underline{x = *p};$$

$$*p = 20 + 1 \Rightarrow 21$$

$$x = 21$$



Uses of pointers (Imp)

Some of the uses of pointers are- -

(i) Accessing array elements.

(ii) Returning more than one value from a function. (Call by Reference)

(iii) Accessing dynamically allocated memory. (Dynamic Memory Allocation)

(iv) Implementing data structures like linked lists, trees, and graphs.



Types of Pointers:

1. Void Pointer
2. NULL Pointer
3. Dangling Pointer
4. Wild Pointer

void Pointer: A pointer to void is a generic pointer that can point to any data type.

```
int a=3, *p;  
float b=4.5, *q;  
p=&a; ✓  
q=&a; ✗
```

```
int a=3, *p;  
void *q;  
p=&a; ✓  
q=&a; ✓
```


Example

/*Program to understand the dereferencing of a void pointer*/

```
#include<stdio.h>
```

```
main( )
```

```
{
```

```
    int a=3; ✓
```

```
    void *vp;
```

```
    vp=&a; ✓
```

```
    printf("Value of a = %d\n", *(int *)vp);
```

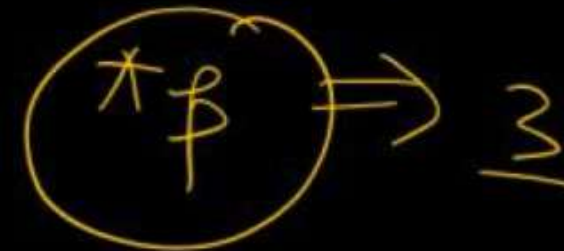
// 3

```
}
```

*vp X

int a=3, *p;

p=&a;



void pointer has to be
typecasted into particular
type of pointer in dereferencing operation.

```
int main()
```

```
{
```

```
    int a=12;
```

```
    void *p=(int*)&a;
```

```
    printf("%d", *p);
```

```
    getchar();
```

```
    return 0;
```

```
}
```

→ compilation error

→ has to be typecasted to (int*)

NULL Pointer:

NULL Pointer is a pointer which is pointing to nothing. In case, if we don't have address to be assigned to a pointer, then we can simply use NULL.

$\text{int}^* p = 0;$

OR

$\text{int}^* p = \underline{\text{NULL}};$

Null pointer

Difference Between Null and Void Pointer

S. No.	Null Pointer	Void Pointer
1	Null represents an empty variable value	A Void denotes a pointer with no initial size.
2	Null means never existed.	Void means, it exists but is no longer in use.
3	This doesn't point to a valid location.	This can point to any type of data.

Dangling pointer:

A pointer pointing to a memory location that has been deleted (or freed) is called dangling pointer.

Example: // Deallocating a memory pointed by ptr causes dangling pointer //

```
#include <stdlib.h>
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int *ptr = (int *)malloc(sizeof(int));
```

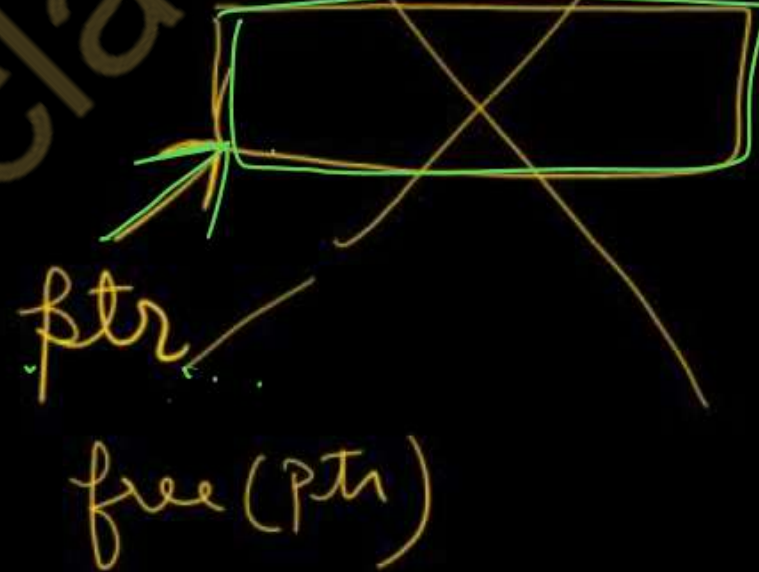
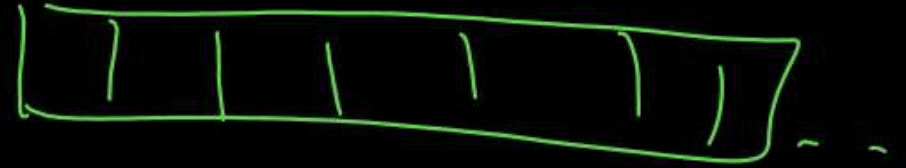
```
free(ptr);
```

// ptr is a dangling pointer

```
ptr = NULL;
```

```
}
```

int a[10];



free(ptr)

// ptr has to be assigned to some valid address or NULL after use.

Wild pointer:

A pointer which has not been initialized to anything (not even NULL) is known as wild pointer. The pointer may be initialized to a non-NULL garbage value that may not be a valid address.

int *p;

Pointer to constant:

Pointer to constant can be declared in following two ways.

const int *ptr;

or

int const *ptr;

Both are same

```
int main ( )  
{
```

```
    const int x = 3;
```

```
    const int *ptr = &x;
```

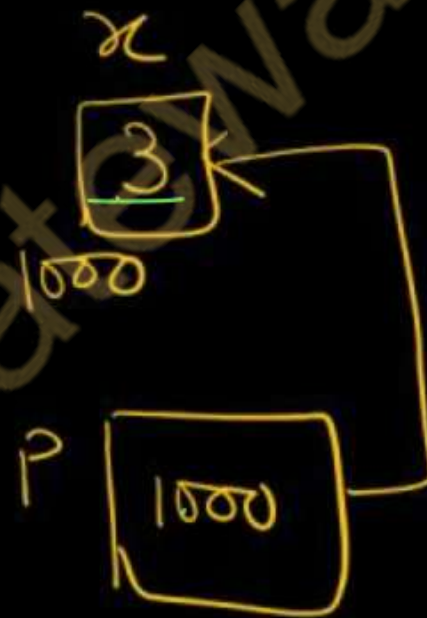
x++; ^{invalid}

ptr++; ✓

ptr = 1000 + 2

= 1002

Valid



Constant pointer to variable:

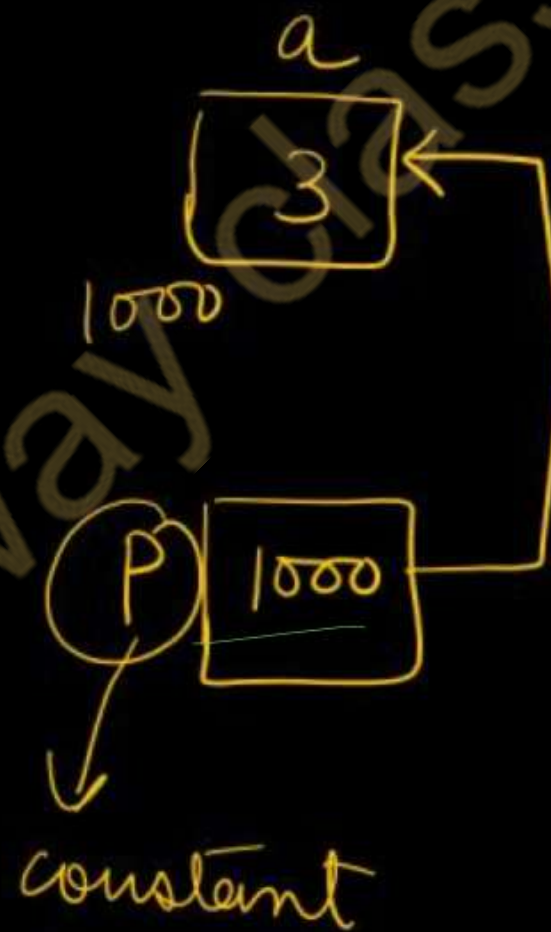
```
int main()  
{ int a=3;  
  int *const p = &a;
```

`a++;` ✓

`p++;` ✗

int *const ptr;
on

const *int ptr;



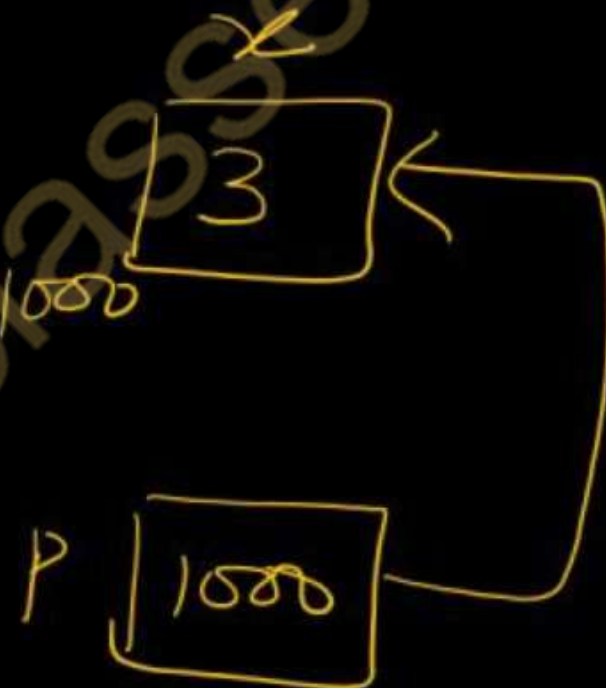
Constant pointer to constant:

const int *const ptr;

```
int main( )  
{  
    const int x=3;  
    const int *const p=&x;
```

`x++;` // Invalid

`p++;` // Invalid



Applications of Pointers in C: (Imp)

1. To pass arguments by reference.
2. For accessing array elements.
3. To return multiple values.
4. Dynamic memory allocation
5. To implement data structures.
6. To do system level programming where memory addresses are useful.

Advantages of pointers:

- (i) Reduce program length.
- (ii) Pointers are helpful in dynamic memory allocation.
- (iii) Pointers enhance the execution speed of a program.
- (iv) Pointers are helpful in traversing through arrays and character strings.
- (v) Pointers also act as references to different types of objects such as variables, arrays, functions, structures etc.

Disadvantages of pointers: (AKTU)

- 1) Uninitialized error (wild pointer)
- 2) Memory leak:

Memory leak occurs when memory is dynamically allocated but not released when it is not required.

- 3) Dangling pointer

- 4) Memory corruption:

Memory corruption often occurs when due to programming errors, the content of a memory location gets modified unintentionally.



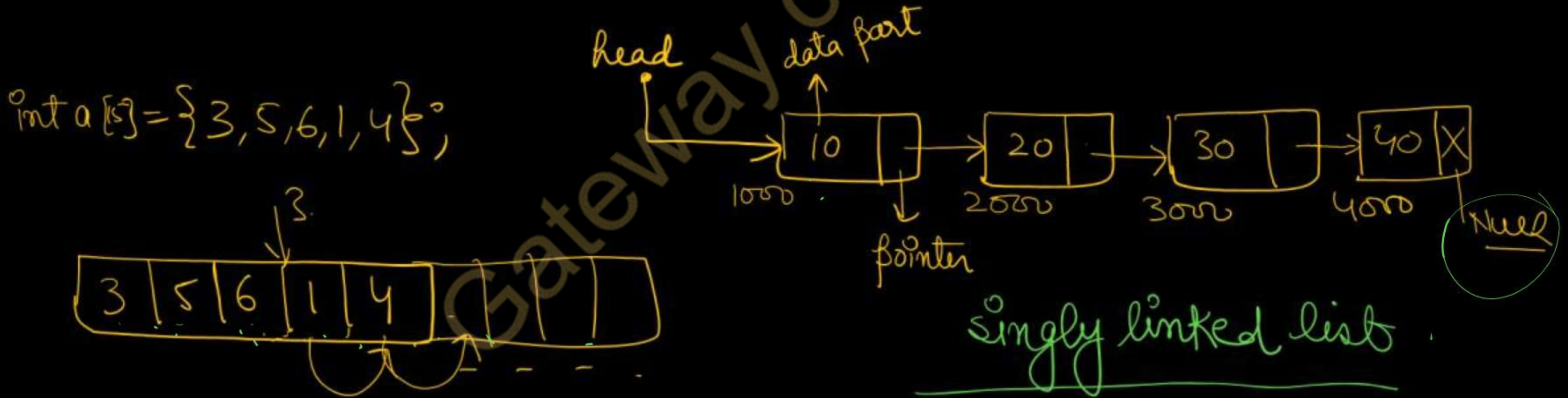
Linked Lists:

A linked list is a dynamic data structure where each element (called a **node**) is made up of two items –

- the data
- pointer which points to the next node.

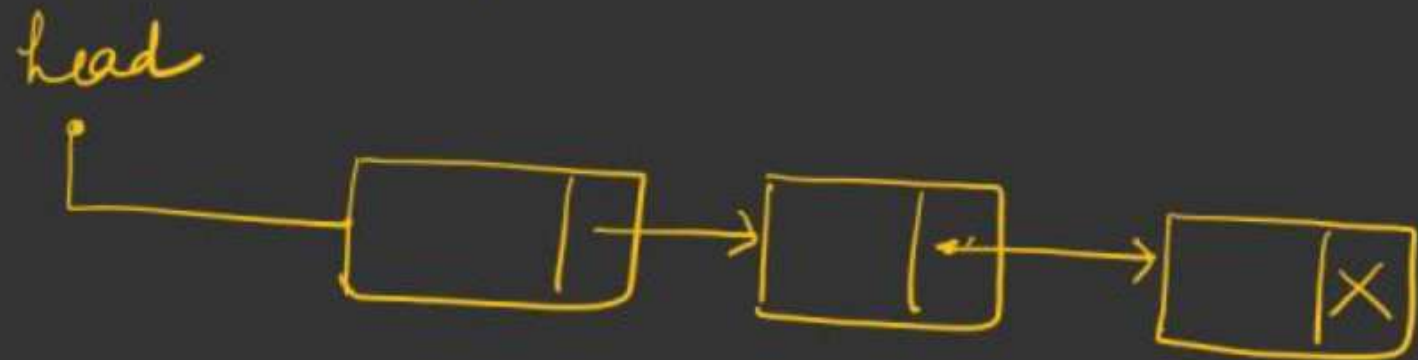
A linked list is a collection of nodes where each node is connected to the next node through a pointer.

The first node is called a **head** ^{pointed by head pointer} and if the list is empty then the value of head is NULL.

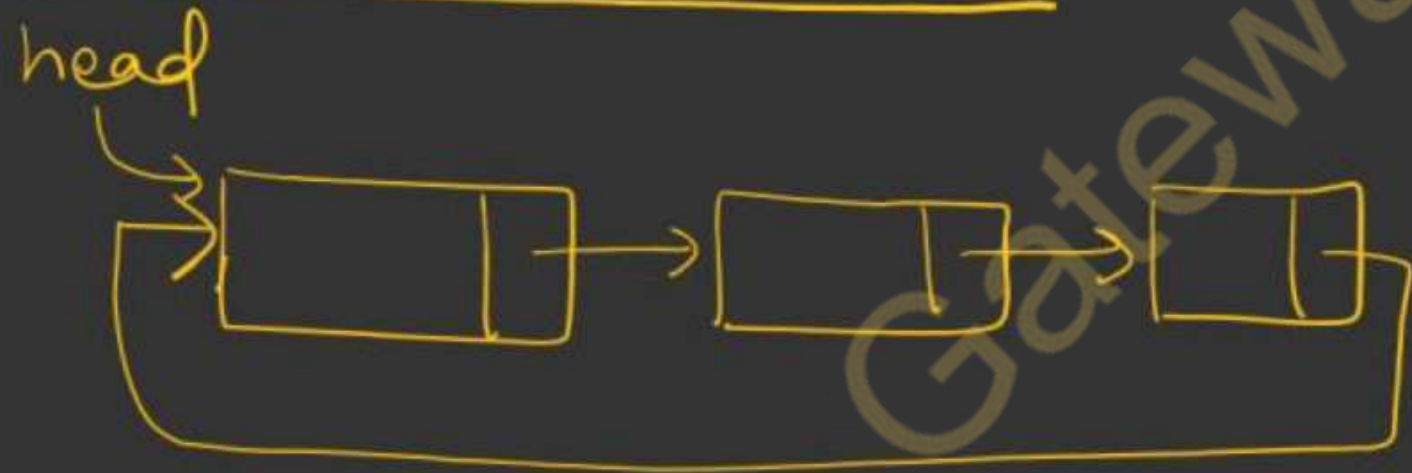


Types of linked lists

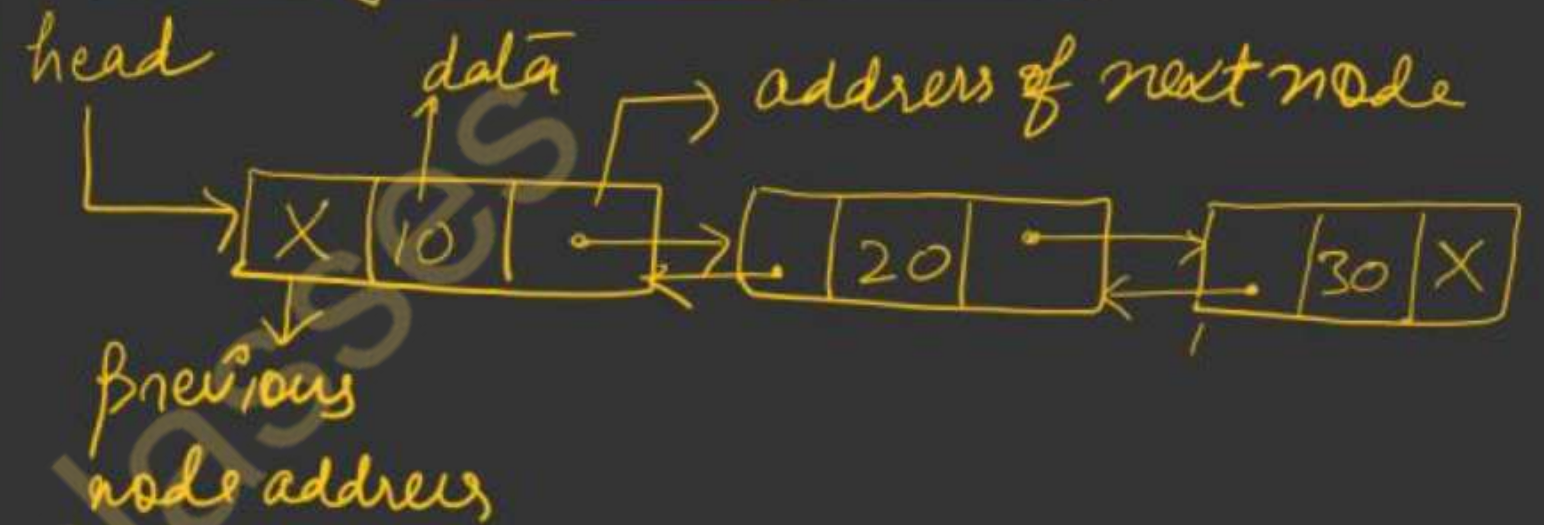
① Singly linked list



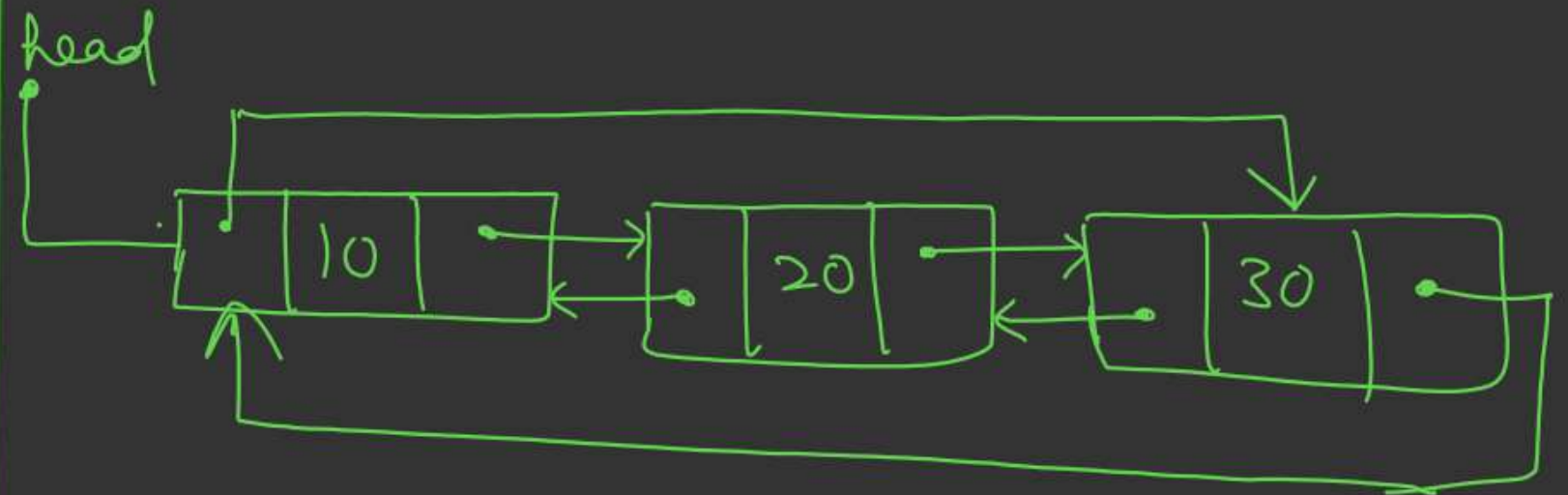
② Circular linked list



③ Doubly linked list



④ Doubly circular linked list



Linked list as Self-referencing Structure:

It is the Data Structure in which the pointer refers (points) to the structure of the same type. It is used in many of the data structures like in Linked List, Trees, Graphs, Heap etc.

Structure of a node

Self
Referential
Structure

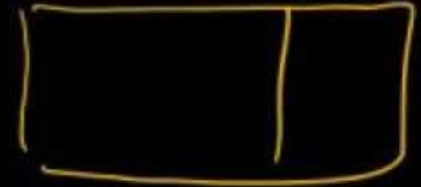
struct node

{
int data;

struct node *next;

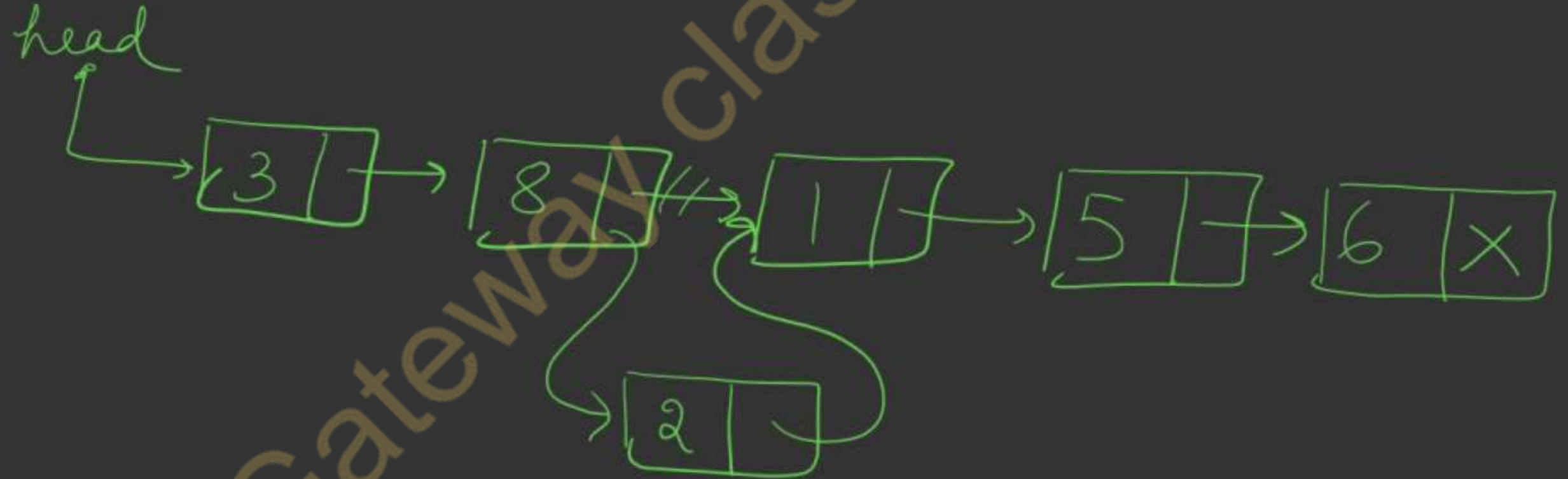
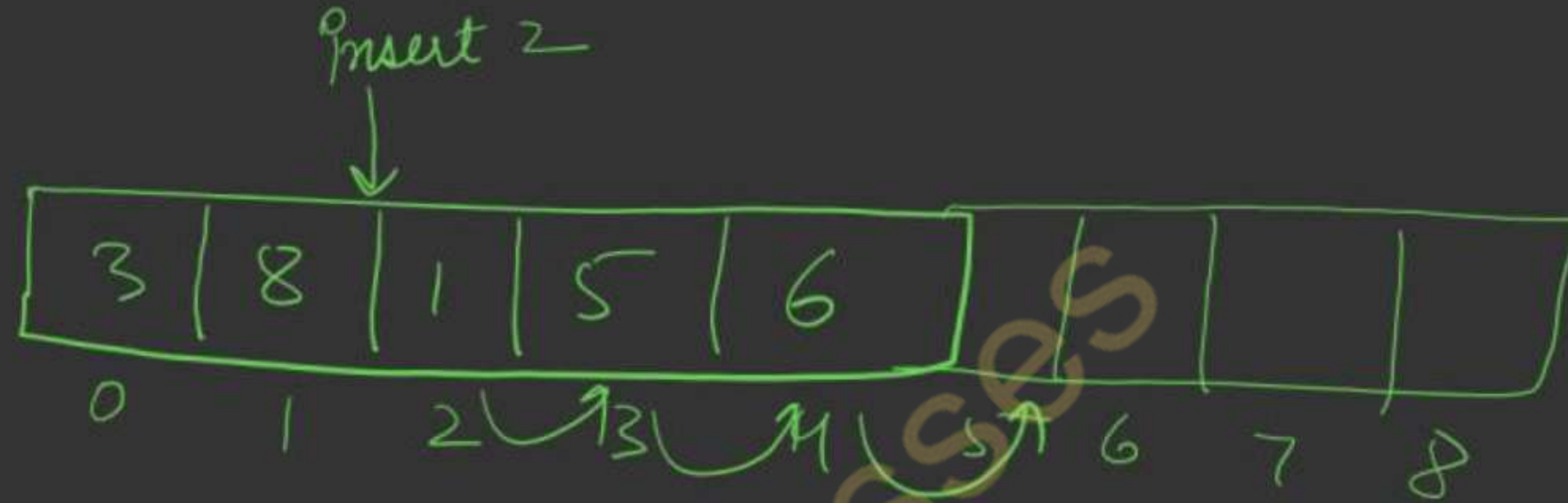
}
x;

x is a node of linked list which will have data and pointer.



Difference between array and linked list

No.	Array	Linked List
1.	Set of fixed number of data items. 	Set of a variable number of data items.
2.	They are stored in contiguous memory locations.	They are not stored in contiguous memory locations.
3.	Memory allocation is done at compile time.	Memory allocation is done at run time.
4.	Arrays are fixed in size.	Linked lists are dynamic in size.
5.	Require less memory space as compared to linked lists.	Linked lists require more memory space.
6.	Insertion and deletion operations require more time to execute.	Insertion and deletion operations take less time.
7.	In arrays, accessing the elements is easier.	Whole linked list is to be traversed to access the elements.



AKTU PYQs

1. Write short note on linked List.(AKTU 2023-24,2022-23)
2. Draw block diagram to represent doubly linked list.(AKTU 2023-24)
3. Define structure of a node in linked list.(AKTU 2022-23, 2018-19)
4. What do you mean by pointer arithmetic.(AKTU 2022-23, 2019-20)
5. List advantage of using a linked list.(AKTU 20220-21)
6. State features of a pointer.(AKTU 2019-20)
7. What is a linked list? Write the self-referential structure of a node of a linked list.(AKTU 2018-19)



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-5

Lecture-3

Topics to be Covered

- Pointer with Array
- Dynamic memory allocation
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Video Lectures | Pdf Notes | Hand Written Notes | DPP

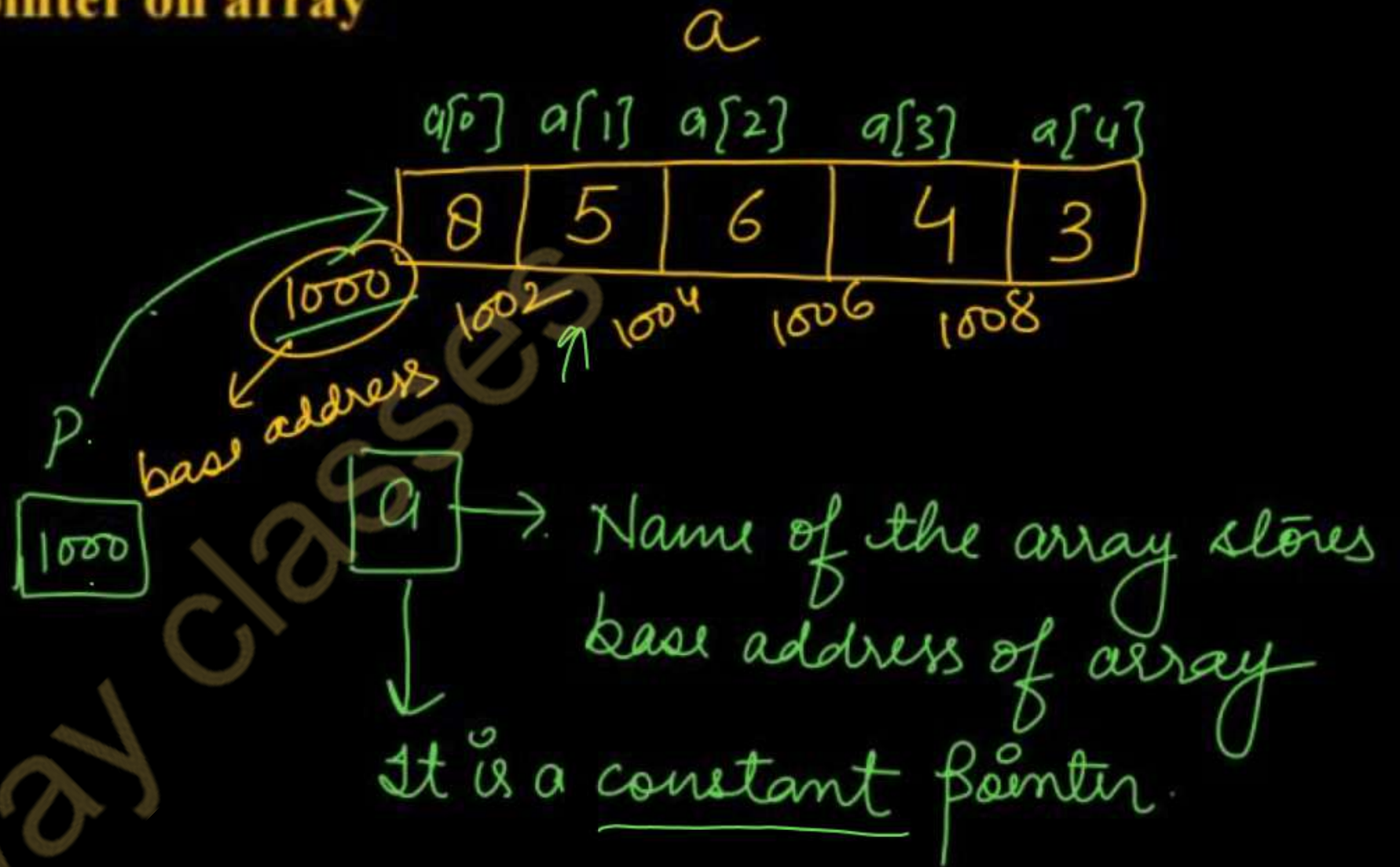
Pointer on array

```
int main()
{
    int a[] = {8, 5, 6, 4, 3}, i;
    int *p;
    p = a; // p = &a[0];

    for(i = 0; i <= 4; i++)
    {
        printf("%d", *p);
        p++;
    }
}
```

p 1000

i = 0	→ p = 1000	*p = 8
i = 1	p = 1002	*p = 5
i = 2	p = 1004	*p = 6
i = 3	p = 1006	*p = 4
i = 4	p = 1008	*p = 3



output

8 5 6 4 3

Pointer on array

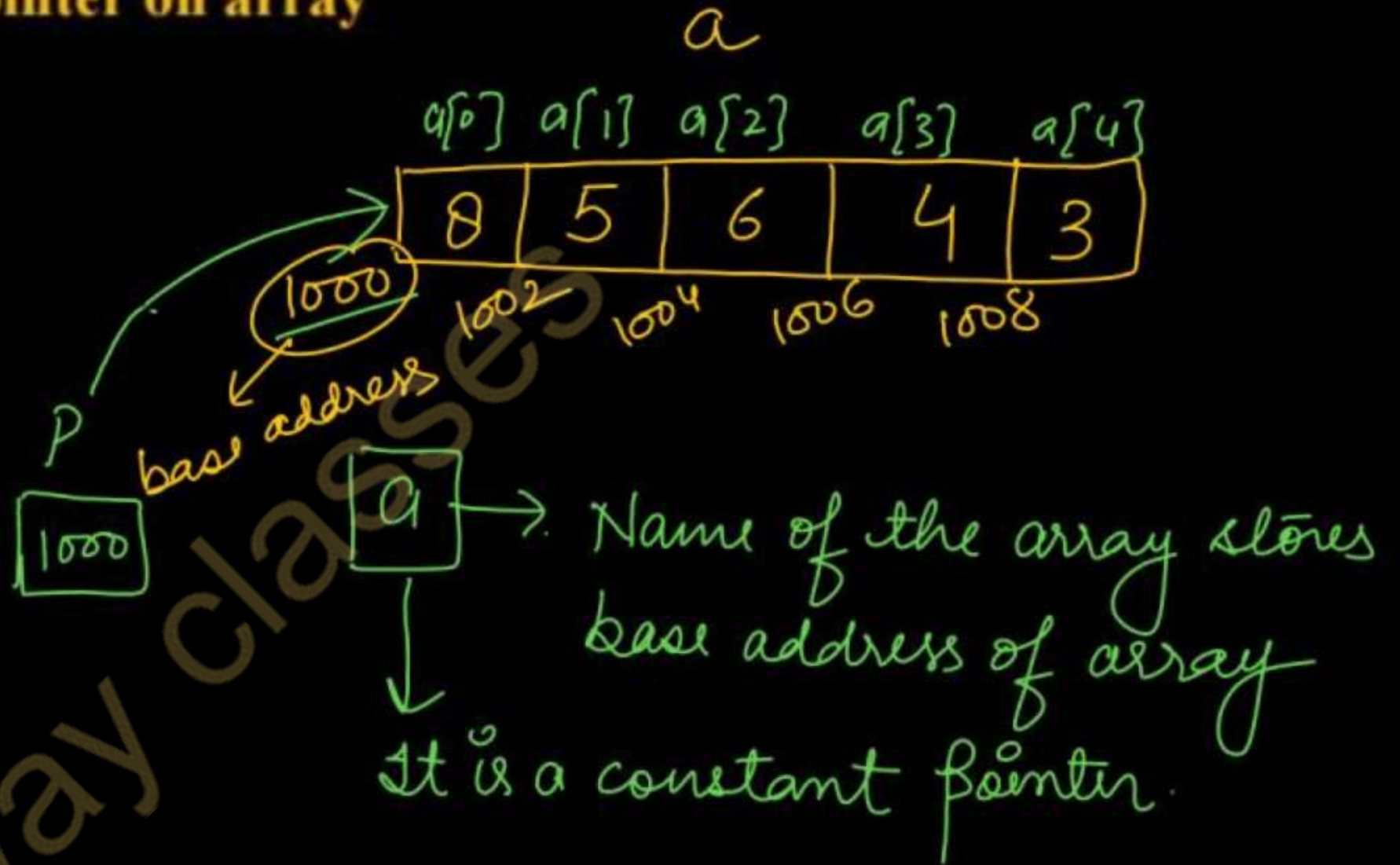
```
int main()
{
    int a[] = {8, 5, 6, 4, 3}, i;
    int *p;
    p = a; // p = &a[0];

    for(i = 0; i <= 4; i++)
    {
        printf("%d", *(p+i));
    }
}
```

* (p+0)

* (p+1)

* (p+2)



output

8 5 6 4 3


```

int main()
{
    int a[5], i, *p, s=0;
    p = &a[0];

```

```

    for (i=0; i<=4; i++)
    {
        scanf("%d", (p+i));
        s = s + *(p+i);
    }

```

```

    printf("The sum of all array elements  
is %d", s);

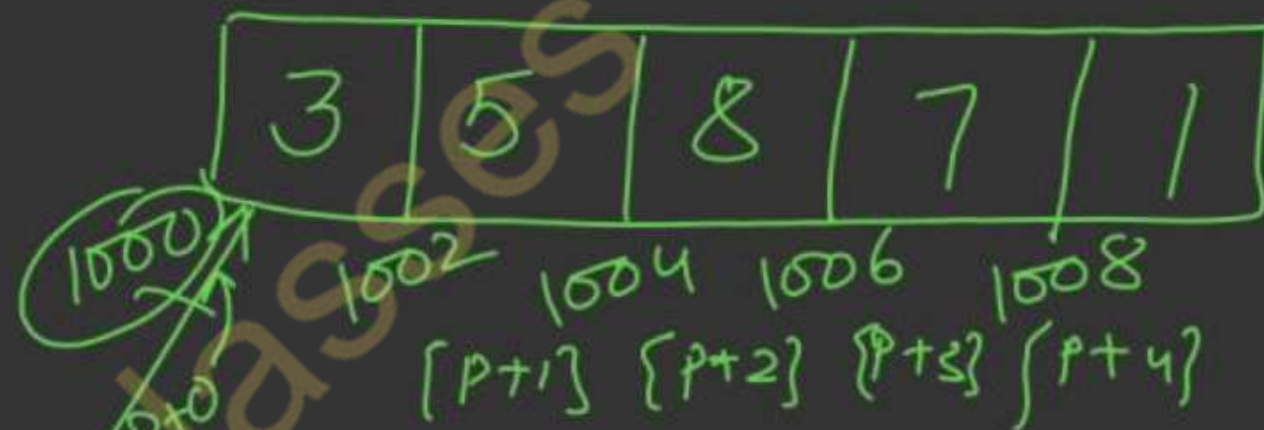
```

```

    return 0;
}

```

a



i=0	i=1
p+0=(1000)	(p+1) → 1002
s=0+(3)	s=3+5

```

int main()
{
    int a[5], i, s=0;
    for(i=0; i<=4; i++)
    {
        scanf("%d", (a+i));
        s = s + *(a+i);
    }
}

```

equivalent to $a[i]$;
 equivalent to $\&a[i]$
 ~~$a++$~~

3	5	1	8	2
1000	1002	1004	1006	1008

$(\underline{a}+0) = 1000 \Rightarrow \&a[0]$

$(\underline{a}+1) = 1002 \Rightarrow \&a[1]$

$(\underline{a}+2) = 1004 \Rightarrow \&a[2]$

$(\underline{a}+3) = 1006 \Rightarrow \&a[3]$

$(\underline{a}+4) = 1008 \Rightarrow \&a[4]$

$(\underline{a}+i) \Rightarrow \&a[i]$

$*(a+i) = a[i]$

$a[0]$	$a[1]$	$a[2]$	$a[3]$	$a[4]$
3	5	1	8	2
1000	1002	1004	1006	1008

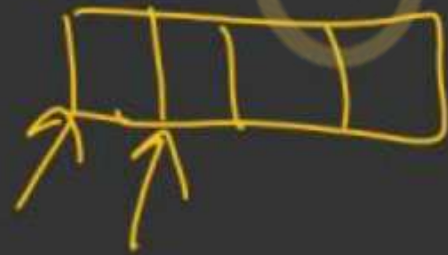
$a+0 \Rightarrow \&a[0]$
 $a+1 \Rightarrow \&a[1]$

$(a+i) \Rightarrow \&a[i]$

$\textcircled{a++}$ $\times$ invalid

constant pointer

$a++$
 $\Rightarrow \underline{\underline{a}} = a+1$



$*(a+0) = a[0]$

$*(a+1) = a[1]$

$*(a+i) = a[i]$

$*(i+a) = i[a]$ ✓

a

`int main()`

`{ int a[] = {1, 2, 3}, i;`

`for(i=0; i<=2; i++)`

`{`

`printf("%d",  $a[i]$ );`

`printf("%d",  $i[a]$ );` same

output 1 1 2 2 3 3

Static Memory Allocation

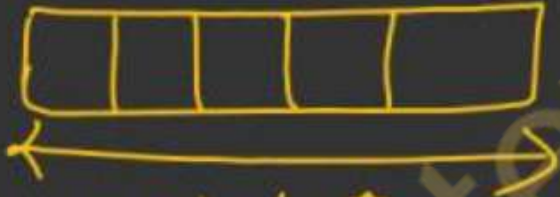
int a = 3;



int a[5];

allocate 10 bytes in advance

int a[100], n;



n $\leftarrow$ 10 $\times$ 2 = 20 bytes

90 $\times$ 2 bytes $\Rightarrow$ 180

Dynamic

malloc()

calloc()

free()

realloc()



Dynamic memory allocation

- An array is a collection of fixed number of values of a single type. That is, we need to declare the size of an array before using it.
- Sometimes, the size of array you declared may be insufficient or too large. To solve this issue, you can allocate memory manually during run-time. This is known as dynamic memory allocation.
- There are 4 library functions defined under `<stdlib.h>` makes dynamic memory allocation in C programming.

1. `malloc()`

2. `calloc()`

3. `realloc()`

4. `free()`

`stdlib.h`

`alloc.h`

`int a[100];`

static Memory Allocation

fixed

⇒ 200 bytes are allocated at compile time.

malloc():

The malloc() function reserves a block of memory of the specified number of bytes.

It returns a pointer of type void which can be casted into pointer of any form.

Syntax of malloc():

ptr = (cast-type*) malloc(byte-size)

Example:

default initial value

garbage value

`int *ptr;`
`ptr = (int *) malloc(10 * sizeof(int));`

Annotations for the example code:

- `int *ptr;`: `int *` is the type, `ptr` is the pointer variable.
- `ptr = (int *) malloc(10 * sizeof(int));`:
 - `(int *)`: Type casting into `int*`.
 - `malloc`: library function.
 - `10`: no. of integers.
 - `sizeof(int)`: Memory required to store one integer value.
 - `10 * sizeof(int)`: 40 bytes.

Considering the size of int is 4 bytes, this statement allocates 40 bytes of memory. And, the pointer ptr holds the address of the first byte in the allocated memory. However, if the space is insufficient, allocation fails and returns a NULL pointer.

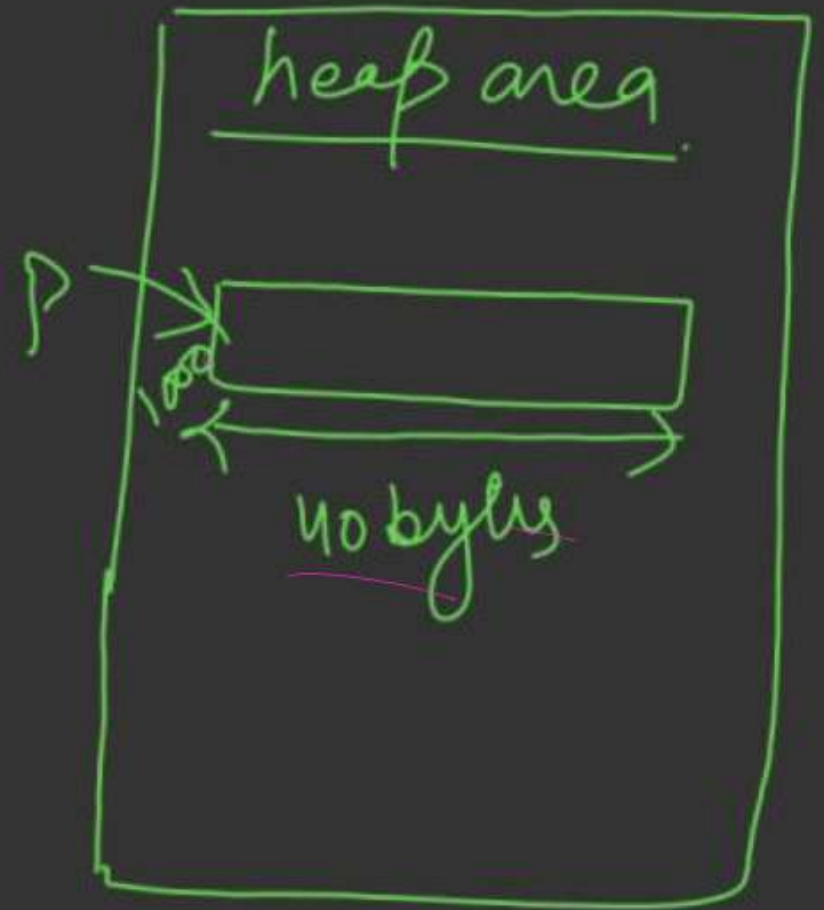
int* p;

p = (int*) malloc (10 * sizeof (int));

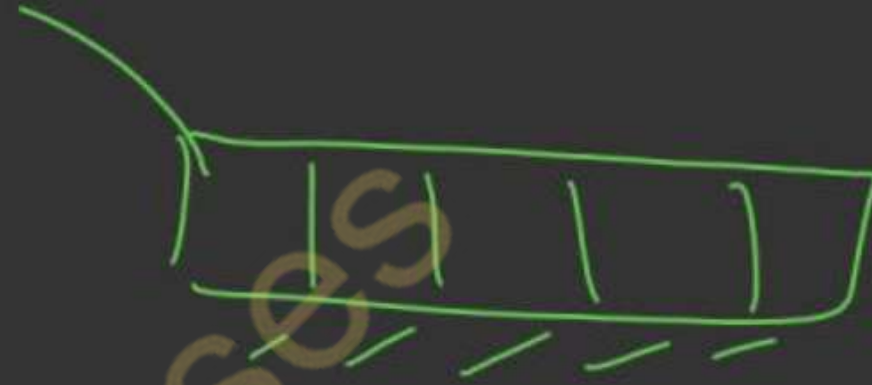
Return type of malloc() and calloc() is void*

On Unsuccessful allocation
malloc and calloc return
NULL.

It will search for
consecutive 40 bytes into
RAM heap area



```
char *q;  
int *p;  
float *r;  
p = (int *) malloc(5 * sizeof(int));  
q = (char *) malloc(4 * sizeof(char));  
r = (float *) malloc(3 * sizeof(float));
```



calloc():

The malloc() function allocates a single block of memory. Whereas, calloc() allocates multiple blocks of memory and initializes them to zero. *It returns a (void*) which has to be typecasted.*

Syntax of calloc():

ptr = (cast-type*)calloc(n, element-size);

float ptr;*

Example:

ptr = (float) calloc(25, sizeof(float));*

no. of blocks
size of one block
contiguous
It will allocate 25 blocks of 4 bytes into the memory

This statement allocates contiguous space in memory for 25 elements each with the size of float.

default initial value $\Rightarrow$ 0

Imp Difference between malloc() and calloc()

malloc()	calloc()
Allocates single block of requested memory	Allocates multiple blocks of requested memory
<pre>int *ptr ptr= malloc(20* sizeof(int));</pre> For the above, <u>20*4</u> bytes of memory only allocated in <u>one block</u>. Total= <u>80 bytes</u>	<pre>int *ptr ptr= calloc(20, sizeof(int));</pre> For the above, 20*4 bytes of memory only allocated in <u>20 blocks</u>. Total=80 bytes
Initializes allocated memory with <u>garbage values</u> .	Initializes the allocated memory <u>with zeros</u> .
Takes one argument ✓	Takes two arguments ✓
type cast must be done since this function returns void pointer <pre>int *ptr; ptr= (int*)malloc(sizeof(int)*20)</pre>	type cast must be done since this function returns void pointer <pre>int *ptr; ptr= (int*)calloc(20*sizeof(int))</pre>

free():

Dynamically allocated memory created with either calloc() or malloc() doesn't get freed on their own. You must explicitly use free() to release the space.

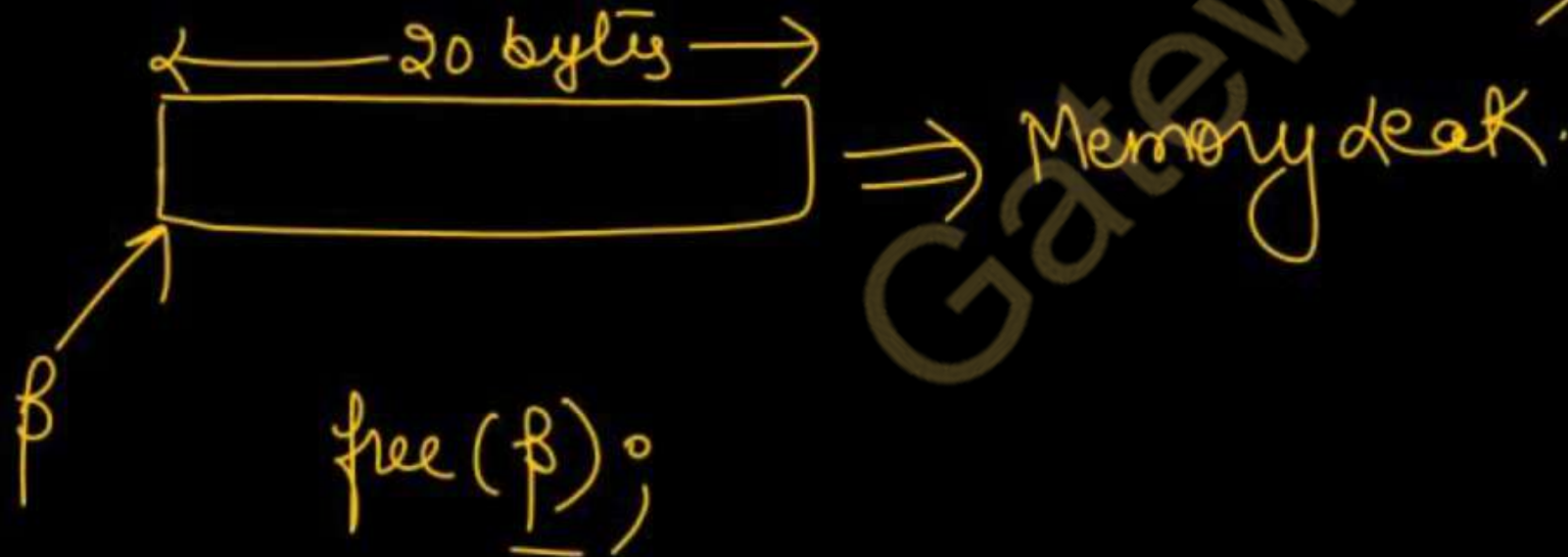
Syntax of free():

free(ptr);

This statement frees the space allocated in the memory pointed by ptr.

`int *p;`

`p = (int*) malloc(5 * 4);`

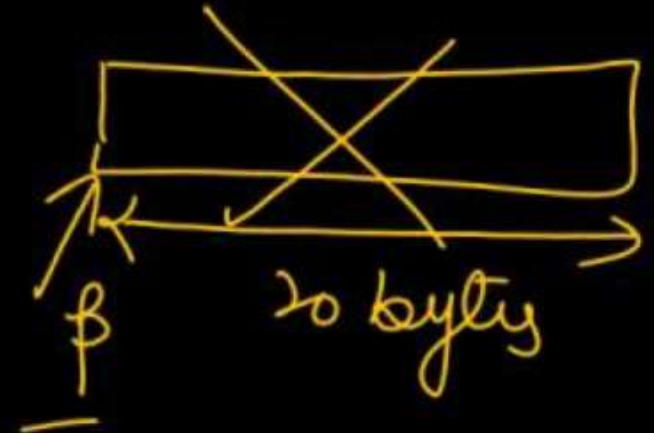


`int *p;`

`p = (int*) malloc(5 * 4);`

`free(p);`

`p = NULL;`



/* Program to print the sum of elements initialized at the dynamic memory allocated by the user */

```
#include <stdio.h>
#include <stdlib.h>
int main()
```

```
{ int n, i, s = 0, *p;
```

Suppose n=5

```
printf("Enter no. of elements");
scanf("%d", &n);
```

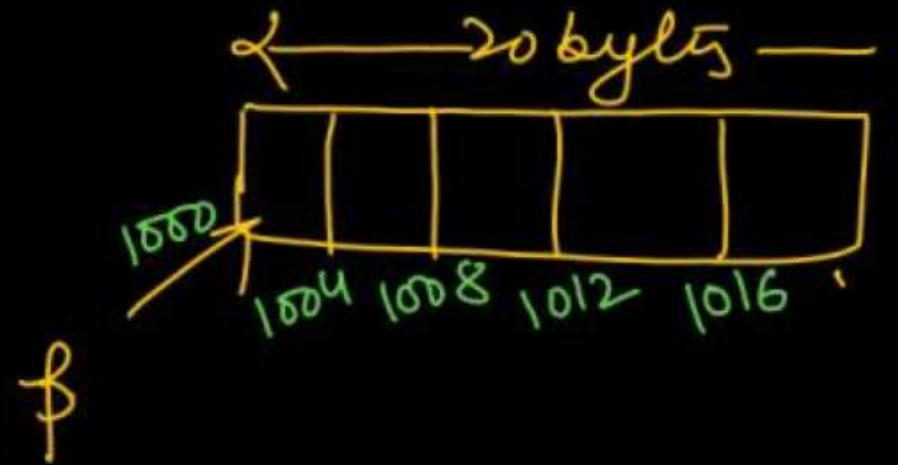
4 bytes

```
p = (int*) malloc(n * sizeof(int));
```

```
if (p == NULL)
```

```
{ printf("Unsuccessful allocation");
```

```
} exit(0);
```



To be contd....

/* Program to print the sum of elements initialized at the dynamic memory allocated by the user */

printf("Enter elements");

for (i=0; i<n-1; i++)

{
scanf("%d", (P+i));

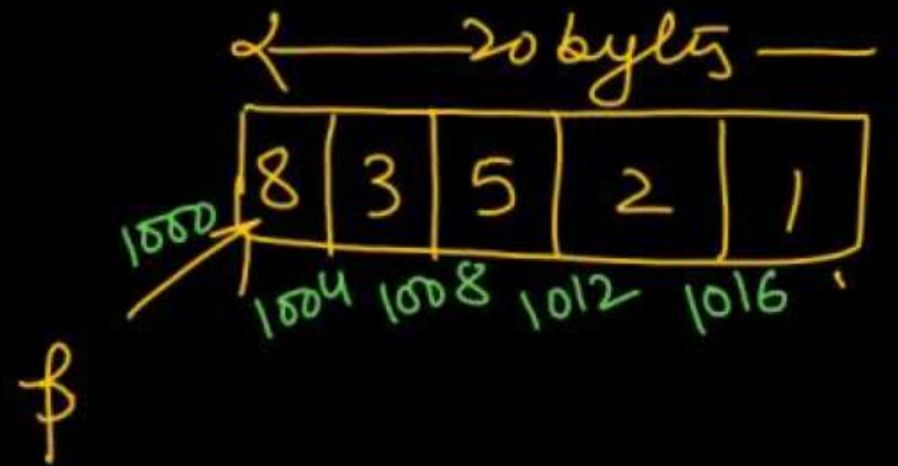
s = s + (* (P+i));

printf("The sum of all elements is = %d", s);

return 0;

}

dynamic array



realloc():

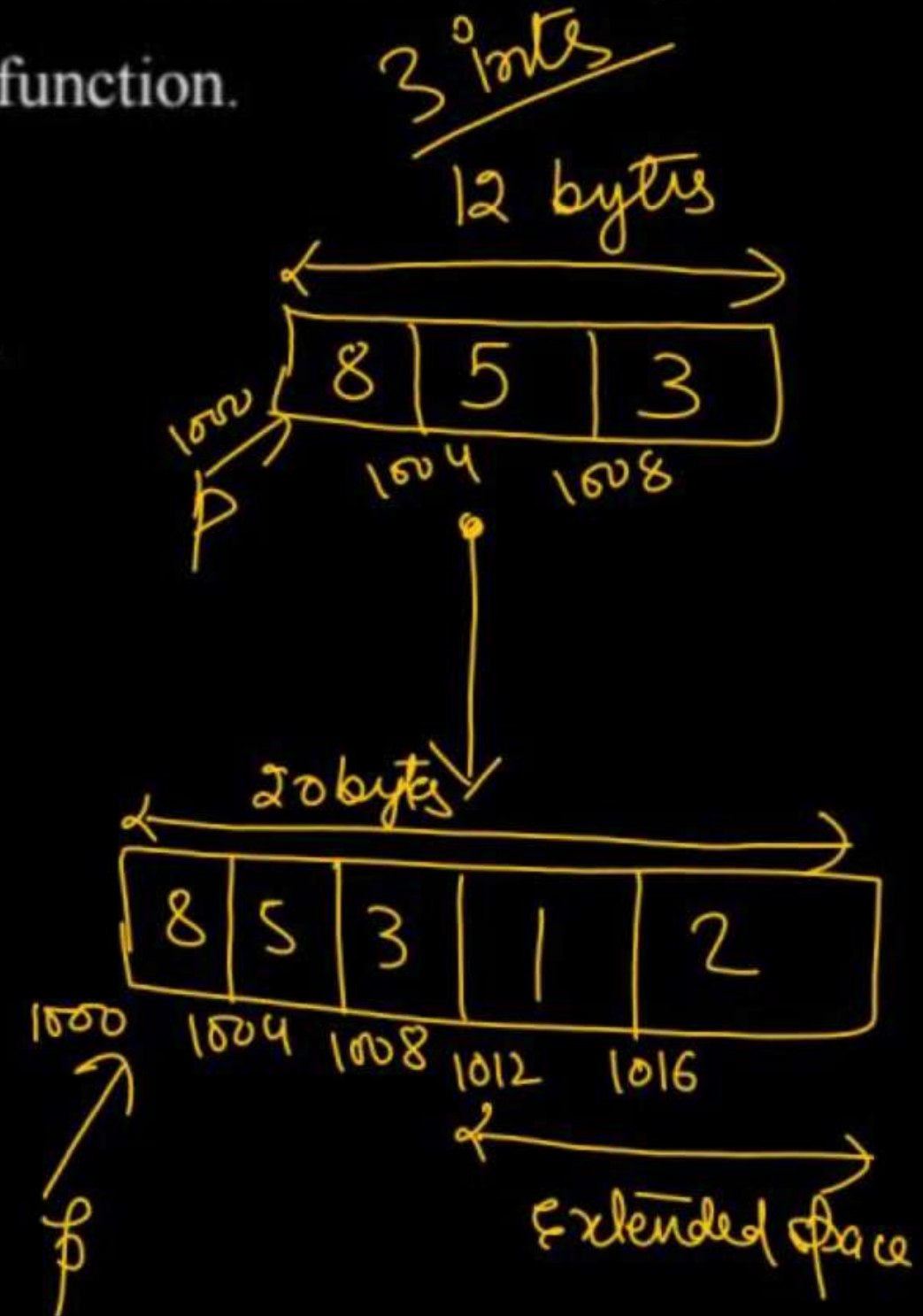
If the dynamically allocated memory using malloc() and calloc() is insufficient or more than required, you can change the size of previously allocated memory using realloc() function.

Syntax of realloc():

ptr = realloc(ptr, x);

Here, ptr is reallocated with new size x.

```
int *p;  
p = (int*) malloc (3 * sizeof(int));  
for (i=0; i<=2; i++)  
{ scanf ("%d", (p+i)); }  
  
p = (int*) realloc (p, 5 * sizeof(int));  
for (i=3; i<=4; i++)  
{ scanf ("%d", (p+i)); }
```

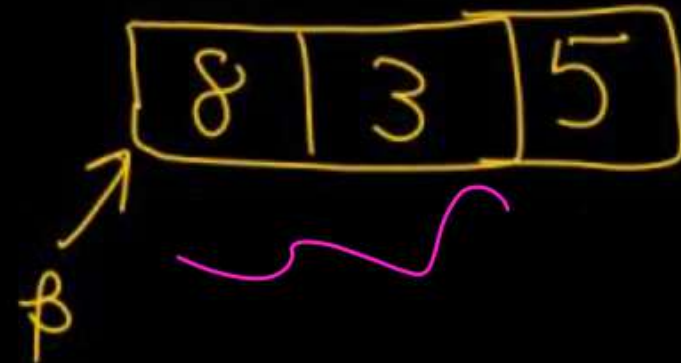


Program to display two numbers with malloc() and reallocate memory for displaying three numbers. (AKTU 2023-24)

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int i, *p;
    p = (int*) malloc(2 * sizeof(int));
    if (p == NULL)
    {
        printf("Unsuccessful allocation");
        exit(0);
    }
    for (i = 0; i <= 1; i++)
    {
        scanf("%d", (p+i));
    }
```

Input
two int
values

```
for (i = 0; i <= 1; i++)
{
    printf("%d", *(p+i));
}
p = (int*) realloc(p, 3 * sizeof(int));
for (i = 2; i <= 2; i++)
{
    scanf("%d", (p+i));
}
for (i = 0; i <= 2; i++)
{
    printf("%d", *(p+i));
}
return 0;
}
```



Difference between static and dynamic memory allocation

S.No	Static Memory Allocation	Dynamic Memory Allocation
1	Allocation of memory at the compile time	Memory allocation at the execution or run time
2	During program execution, memory cannot be changed.	During program execution, memory can be changed.
3	Static memory allocation is preferred in an array.	Dynamic memory allocation is preferred in linked list.
4	It saves running time as it is fast.	It is slower than static memory allocation.
5	Once the memory is allotted, it will automatically be released after execution of the program according to scope rules.	Here, the memory has to be deallocated explicitly using free function.
6	Static memory allocation is less efficient	Dynamic memory allocation is more efficient

AKTU PYQs

1. Define calloc() function. Write a program to print the sum of elements initialized at the dynamic memory allocated by the user.(AKTU 2023-24,2022-23)
2. What are the advantages of dynamic memory allocation over static memory allocation? Write a program to display two numbers with malloc() and reallocate memory for displaying three numbers.(AKTU 2023-24)
3. Discuss dynamic memory allocation. Explain calloc(),malloc(),realloc(), and free() functions with suitable examples.(AKTU 2022-23)
4. Explain static memory allocation and dynamic memory allocation with suitable examples.(AKTU 2022-23, 2018-19))
5. What is the requirement of free() function in dynamic memory allocation?(AKTU 2022-23)
6. Define and explain dynamic memory allocation. Differentiate malloc() and calloc() functions with proper example.(AKTU 2021-22,2019-20)



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-5

Lecture-4

Topics to be Covered

- File Handling
- Command line arguments
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Video Lectures | Pdf Notes | Hand Written Notes | DPP

File Handling

File handling in C enables us to create, update, read, and delete the files stored on the local file system through our C program.

The following operations can be performed on a file.

- Creation of the new file
- Opening an existing file
- Reading from the file
- Writing to the file
- Deleting the file

Why do we need File Handling in C?

- ① **Reusability:** The data stored in the file can be accessed, updated, and deleted anywhere and anytime providing high reusability.
- ② **Portability:** Without losing any data, files can be transferred to another in the computer system. The risk of flawed coding is minimized with this feature.
- ③ **Efficient:** A large amount of input may be required for some programs. File handling allows you to easily access a part of a file using few instructions which saves a lot of time and reduces the chance of errors.
- ④ **Storage Capacity:** Files allow you to store a large amount of data without having to worry about storing everything simultaneously in a program.

Types of Files in C

A file can be classified into two types based on the way the file stores the data. They are as follows:

✓ **Text Files**

✓ **Binary Files**

1. Text Files(.txt file extension)

A text file contains data in the **form of ASCII characters** and is generally used to store a stream of characters. Each line in a text file ends with a new line character ('\n').

2. Binary Files(.bin file extension)

A binary file contains data in **binary form (i.e. 0's and 1's)**. They contain data that is stored in a similar manner to how it is stored in the main memory.



'\n'

#include <stdio.h>

FILE *fp;

fp = fopen ("A.txt", "r");

FILE

in-built structure
in stdio.h

file name

mode in which
file opening
request is made

A.txt

Gateway classes

if (fp == NULL)
{

}

fclose(fp); char filename[50];
gets(filename);

fopen(filename, "r");

File Pointer in C

- A file pointer is a reference to a particular position in the opened file.
- It is used in file handling to perform all file operations such as read, write, close, etc.
- We use the FILE ^{structure} ~~macro~~ to declare the file pointer variable.
- The FILE ~~macro~~ is defined inside <stdio.h> header file.

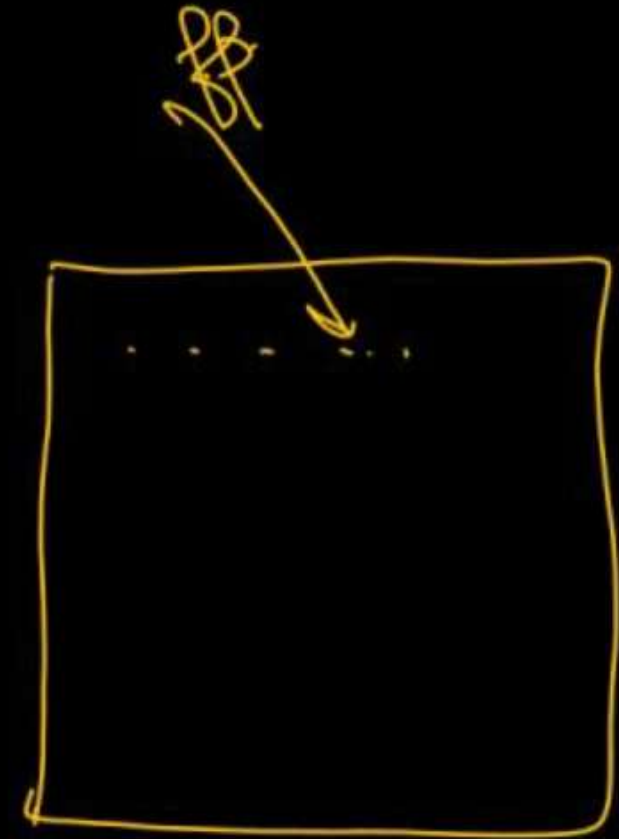
Syntax of File Pointer

FILE* *pointer_name*;

typedef long int li;

typedef struct {

} FILE;



Functions for C File Operations

Opening File: fopen()

- We must open a file before it can be read, write, or update.
- The fopen() function is used to open a file.
- fopen() function creates a new file if the mentioned file name does not exist.

FILE *fp

fp = fopen ("filename", "mode");

"r" / "w"
NULL / new file is created.

Note — It returns NULL if file does not exist. *in read mode*

- fp: is file pointer to the data type "FILE"
- filename: the actual file name with full path of the file.
- mode: operation that will be performed on the file such as r, w, a, r+, w+, a+.

File Opening Modes (V. Imp)

Mode	Meaning of Mode
<u>r</u>	Searches file. If the file is opened successfully fopen() loads it into memory and sets up a pointer which points to the first character in it. If the file cannot be opened fopen() returns NULL. <i>If requested file does not exist then also it returns NULL.</i>
<u>w</u>	Searches file. If the file exists, existing content will be erased. If the file doesn't exist, a new file is created. Returns NULL, if unable to open file.
<u>a</u>	Searches file. If the file is opened successfully fopen() loads it into memory and sets up a pointer that points to the last character in it. If the file doesn't exist, a new file is created. Returns NULL, if unable to open file

File Opening Modes

Mode	Meaning of Mode
<u>r+</u>	Searches file. If is opened successfully fopen() loads it into memory for <u>both reading and writing</u> . It does not delete the existing content. Existing contents can be modified anywhere using this mode. Returns NULL, if unable to open the file. <i>if file does not exists $\Rightarrow$ returns NULL</i>
<u>w+</u>	Searches file. If the file exists, its opened for both reading and writing. Existing content will be erased. <u>If the file doesn't exist a new file is created</u> . Returns NULL, if unable to open file.
<u>a+</u>	Searches file. If the file is opened successfully for reading and <u>appending</u> . If the file doesn't exist, a new file is created. Returns NULL, if unable to open file.

Write to a File

The file write operations can be performed by the functions `fprintf()` and `fputs()` with similarities to read operations. C programming also provides some other functions that can be used to write data to a file such as:

Function	Description
✓ <u><code>fprintf()</code></u>	Similar to <code>printf()</code> , this function use <u>formatted string</u> and variable arguments list to print output to the file.
<u><code>fputs()</code></u>	Prints the whole line in the file and a newline at the end.
<u><code>fputc()</code></u>	Prints a single character into the file.
<u><code>fputw()</code></u>	Prints a number to the file.
✓ <u><code>fwrite()</code></u>	This functions write the specified number of bytes to the binary file.

How to write into a file

fputc()

```
#include <stdlib.h>
#include <stdio.h>
int main()
```

```
{
    FILE *fp;
```

```
    char ch = 'A';
```

```
    fp = fopen("abc.txt", "w");
```

```
    if (fp == NULL)
```

```
    {
        printf("Unsuccessful Attempt  
to open the file");
```

```
    }
    exit(1);
```

```
    fputc(ch, fp);
```

```
    return 0;
```

```
}
```

abc.txt

A

fputs()

abc.txt

Gateway

char str[] = "Gateway";

fputs(str, fp);

fprintf()

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    FILE *fptr ;
```

```
    fptr = fopen("fileName.txt", "w");
```

```
    fprintf(fptr, "%s %s %s %d", "We", "are", "in", 2025);
```

```
    fclose(fptr);
```

```
    return 0;
```

```
}
```

file Name.txt

We are in 2025

Reading From a File

The file read operation in C can be performed using functions `fscanf()` or `fgets()`.

Both the functions perform the same operations as that of `scanf` and `gets` with an additional parameter, the file pointer. The functions we can use to read from a file. Such functions are listed below:

Function	Description
✓ <code>fscanf()</code> ✓	Use formatted string and variable arguments list to take input from a file.
✓ <code>fgets()</code> ✓	Input the whole line from the file.
✓ <code>fgetc()</code> ✓	Reads a single character from the file.
✓ <code>fgetw()</code> ✓	Reads a number from a file.
✓ <code>fread()</code>	Reads the specified bytes of data from a binary file.

How to read from a file

```
#include <stdio.h>
```

```
int main( )
```

```
{
```

```
    FILE *fp; char ch;
```

```
    fp = fopen("abc.txt", "r");
```

```
    if (fp == NULL)
```

```
    {  
        //  
    }
```

```
    ch = fgetc(fp);
```

```
    printf("%c", ch); // 5
```

EOF



```
ch = fgetc(fp); ✓  
fputc(ch, fp);
```

Reading a File in C using fgetc()

```
#include <stdio.h>
```

```
int main()
```

```
{ FILE *fp;
```

```
    fp = fopen("A.txt", "r");
```

```
    if (fp == NULL)
```

```
    {
```

```
        printf("Error: Could not open file.\n");
```

```
        exit(0);
```

```
    }
```

```
    printf("File contents:\n");
```

```
    while ((ch = fgetc(fp)) != EOF)
```

```
    {
```

```
        printf("%c", ch);
```

```
    }
```

```
    fclose(fp);
```

```
    return 0;
```

```
}
```

EOF

Closing File: fclose()

The fclose() function is used to close a file. The file must be closed after performing all the operations on it.

fclose(FILE *fp)

fclose() function closes the file that is being pointed by file pointer fp.

fclose(fp)

Program in C to write some text into a file "example.txt" and then read this text and display on console using file handling (AKTU 2019-20)

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE * fp;
    char ch;
    fp = fopen("example.txt", "r");
    if (fp == NULL)
    {
        printf("\nUnable to open file.\n");
        exit(0);
    }
```

```
while ((ch = fgetc(fp)) != EOF)
```

```
{
```

```
    printf("%c",ch);
```

```
}
```

```
fclose(fp);
```

```
return 0;
```

```
}
```


Program to read content of an existing file and display numbers of characters, blanks, tabs and newlines in the file..(AKTU 2019-20)

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *fp;
    int noc=0, non=0, net=0, nob=0;
    char ch;
    fp = fopen("A.txt", "r");
    if (fp == NULL)
    {
        printf("File could not be opened");
        exit(0);
    }
}
```

```
while((ch = fgetc(fp)) != EOF)
{
```

```
    noc++;
```

```
    if (ch == '\n')
```

```
        non++;
```

```
    if (ch == '\t')
```

```
        net++;
```

```
    if (ch == ' ')
```

```
        nob++;
```

```
    }
    printf("No. of characters=%d", noc);
```

```
    printf("\nNo. of tabs=%d", net);
```

```
    printf("\nNo. of blanks=%d", nob);
```

```
    printf("\nNo. of newlines are=%d", non);
```

```
    fclose(fp);
```

```
    return 0;
```

Program to read content of an existing file and display numbers of words and lines in that file..(AKTU 2022-23)

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE * file;
    char ch;
    int characters, words, lines;
    file = fopen("A.txt", "r");
    if (file == NULL)
    {
        printf("\nUnable to open file.\n");
        printf("Please check if file exists and you have read privilege.\n");    exit(0);
    }
    characters = words = lines = 0;
```



```
while ((ch = fgetc(file)) != EOF)
{
    characters++;
    if (ch == '\n' || ch == '\0')
        lines++;
    if (ch == ' ' || ch == '\t' || ch == '\n' || ch == '\0')
        words++;
}
if (characters > 0)
{
    words++;
    lines++;
}
printf("Total characters = %d\n", characters);
printf("Total words = %d\n", words);
printf("Total lines = %d\n", lines);
fclose(file);
return 0;
}
```

Hello ✓
Gateway classes ✓

Program in C to write multiple lines in a text file.(AKTU 2023-24)

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
FILE * fptr;
```

```
int i, n;
```

```
char str[100];
```

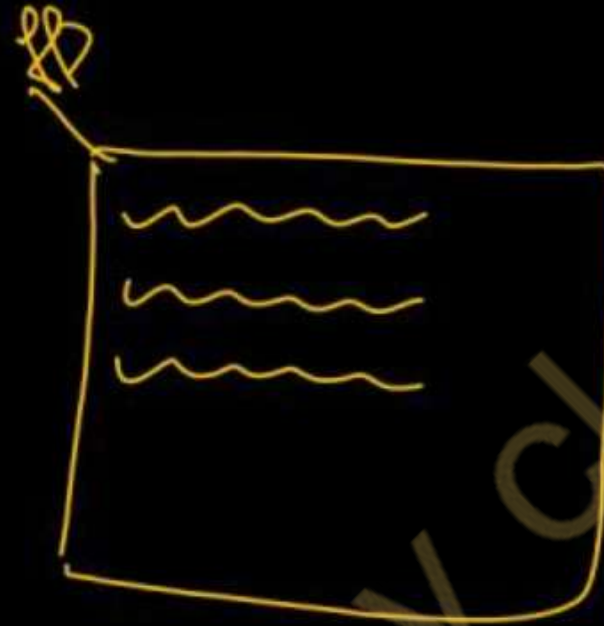
```
char fname[20]="A";
```

```
char str1;
```

```
printf(" Input the number of lines to be written : ");
```

```
scanf("%d", &n);
```

fgets() reads bytes from stream into the array pointed to by str, until a newline character is read and transferred to str



Hello ←

```
printf("\n :: The lines are ::\n");
```

```
fptr = fopen (fname, "w");
```

```
for(i = 0; i < n+1 ; i++)
```

```
{   fgets(str, sizeof str, stdin);
```

```
    fputs(str, fptr);
```

```
}
```

```
fclose (fptr);
```

```
return 0;
```

```
}
```

takes text
from standard
input

Program to copy the contents of one file into another using file handling(AKTU 2021-22)

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *fptr1, *fptr2;
    char filename[100], c;
    printf("Enter the filename to open for reading \n");
    scanf("%s", filename);
    fptr1 = fopen(filename, "r");
    if (fptr1 == NULL)
    {
        printf("Cannot open file %s \n", filename);
        exit(0);
    }
```

V. Imp

```
printf("Enter the filename to open for writing \n");
```

```
scanf("%s", filename);
```

```
fptr2 = fopen(filename, "w");
```

```
if (fptr2 == NULL)
```

```
{  
    fclose(fptr1);  
    printf("Cannot open file %s \n", filename);  
    exit(0);  
}
```

```
while ((c = fgetc(fptr1)) != EOF)
```

```
{  
    fputc(c, fptr2);  
}
```

```
printf("\n Contents copied to %s", filename);
```

```
fclose(fptr1);
```

```
fclose(fptr2);
```

```
return 0;
```

```
}
```

source file

fptr1



target file

fptr2



// both files are Successfully open.

Program to count number of vowels in a text file. (AKTU 2023-24)

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *fp;
    int vowels = 0;
    char ch;
    fp = fopen("A.txt", "r");
    if (fp == NULL)
    {
        printf("File not opened");
        exit(0);
    }
}
```

```
while ((ch = fgetc(fp)) != EOF)
{
    if ((ch == 'a') || (ch == 'e') || (ch == 'i') || (ch == 'o')
        || (ch == 'u') || (ch == 'A') || (ch == 'E') || (ch == 'I')
        || (ch == 'O') || (ch == 'U'))
        vowels++;
}
printf("No. of vowels = %d", vowels);
fclose(fp);
return 0;
}
```

fseek()

It is used to move the position of file pointer.

FILE *fp;

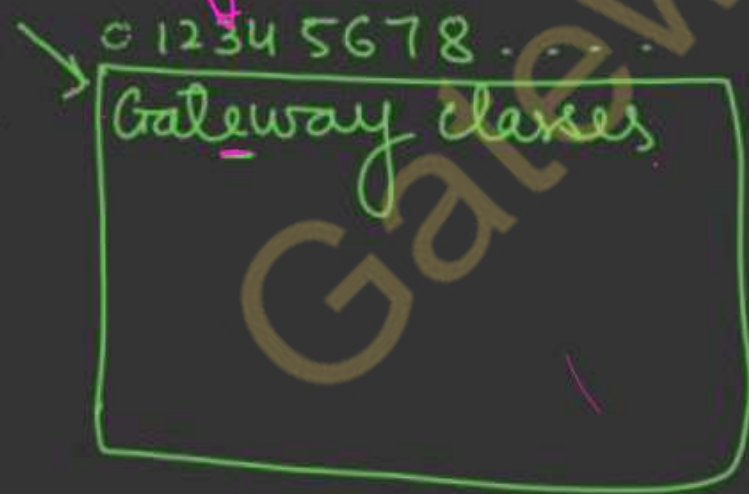
fseek (file pointer, ⁽³⁾ OFFSET, Position);

fp

long int

{ -ve
+ve }

- 1) SEEK-SET → Beginning
- 2) SEEK-CUR → current
- 3) SEEK-END → Ending position



rewind()

rewind(fp);

↓
It brings the file pointer to the beginning of file.

Command Line Arguments in C

- The main() function is the most significant function of C language.
- This main() is typically defined as having a return type of integer and having no parameters

```
int main (    )  
{  
      
      
      
}
```

Command line arguments are the arguments specified after the program name in the operating system's command line.

These argument values are passed to program during execution from operating system.

To pass command-line arguments, we typically define main() with two arguments: the first argument is the **number of command-line arguments** and the second is a **list of command-line arguments**.



```
int main(①int ②argc, char *argv[])  
{  
    /* ... */  
}
```

- **argc** is a non-negative integer variable that stores the number of command-line arguments passed by the user including the name of the program. So, if we pass a value to a program, the value of **argc** would be 2 (one for argument and one for program name).
- **argv** is an array of character pointers listing all the arguments.
- If argc is greater than zero, then argv[0] is the name of the program, After that till argv[argc-1] every element is command -line arguments.

Properties of Command Line Arguments in C

- They are passed to the `main()` function.
- They are parameters/arguments supplied to the program when it is invoked.
- `argv[argc]` is a NULL pointer.
- `argv[0]` holds the name of the program.
- `argv[1]` points to the first command line argument and `argv[argc-1]` points to the last argument.

$argc = 5$

0 — 4

```
#include <stdio.h>
int main(int argc, char* argv[])
{
    printf("Program name is: %s", argv[0]);
    if (argc == 1)
        printf("\nNo Extra Command Line Argument Passed Other Than Program Name");
    if (argc >= 2)
    {
        printf("\nNumber Of Arguments Passed: %d", argc);
        printf("\n----Following Are The Command Line Arguments Passed----");
        for (int i = 0; i < argc; i++)
            printf("\n argv[%d]: %s", i, argv[i]);
    }
    return 0;
}
```


Output in different scenarios:

1. Without argument:

Terminal Input

\$./a.out

Output

Program Name Is: ./a.out

No Extra Command Line Argument Passed Other Than Program Name

2. Three arguments:

Terminal Input

\$./a.out First Second Third

Output

Program Name Is: ./a.out

Number Of Arguments Passed: 4

----Following Are The Command Line Arguments Passed----

argv[0]: ./a.out

argv[1]: First

argv[2]: Second

argv[3]: Third

AKTU PYQs

1. List out various file opening modes .(AKTU 2022-23) _____
2. Discuss various file operations. Write a program in C to read content of an existing file and display the numbers of words and lines in that file .(AKTU 2022-23)
3. Define any three functions used in file handling in C. Write a program to count number of vowels in a text file.(AKTU 2023-24)
4. Define File. Write the modes used in file handling. Write a program in C to write multiple lines in a text file.(AKTU 2023-24)
5. Explain different types of modes and I/O functions used in file handling.(AKTU 2020-21)
6. Differentiate between read and write mode in file handling. Write a program to copy the contents of one file into another using file handling?(AKTU 2021-22)

AKTU PYQs

7. Discuss various file handling methods used in C in brief. Write a program in C to write some text into a file “example.txt” and then read this text and display on console using file handling (AKTU 2019-20)
8. Explain command line arguments in C with the help of an example. (AKTU 2018-19)

Thank
you